

Object detection and segmentation

Rikke Gade 2026

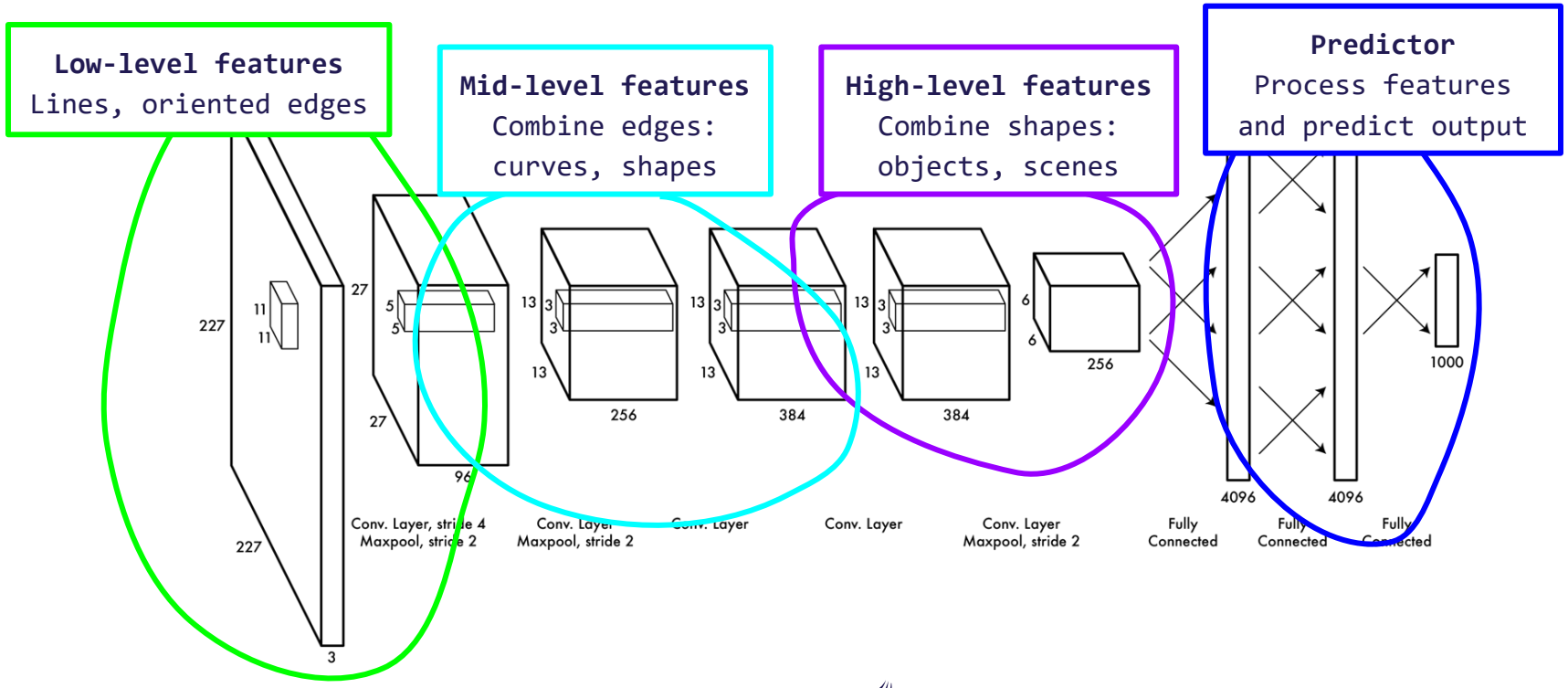
AALBORG
UNIVERSITY

Agenda

- Brief CNN/Transformers recap
- Applications of CNNs
  - Object detection
  - Semantic segmentation
  - Instance segmentation



Understanding neural networks



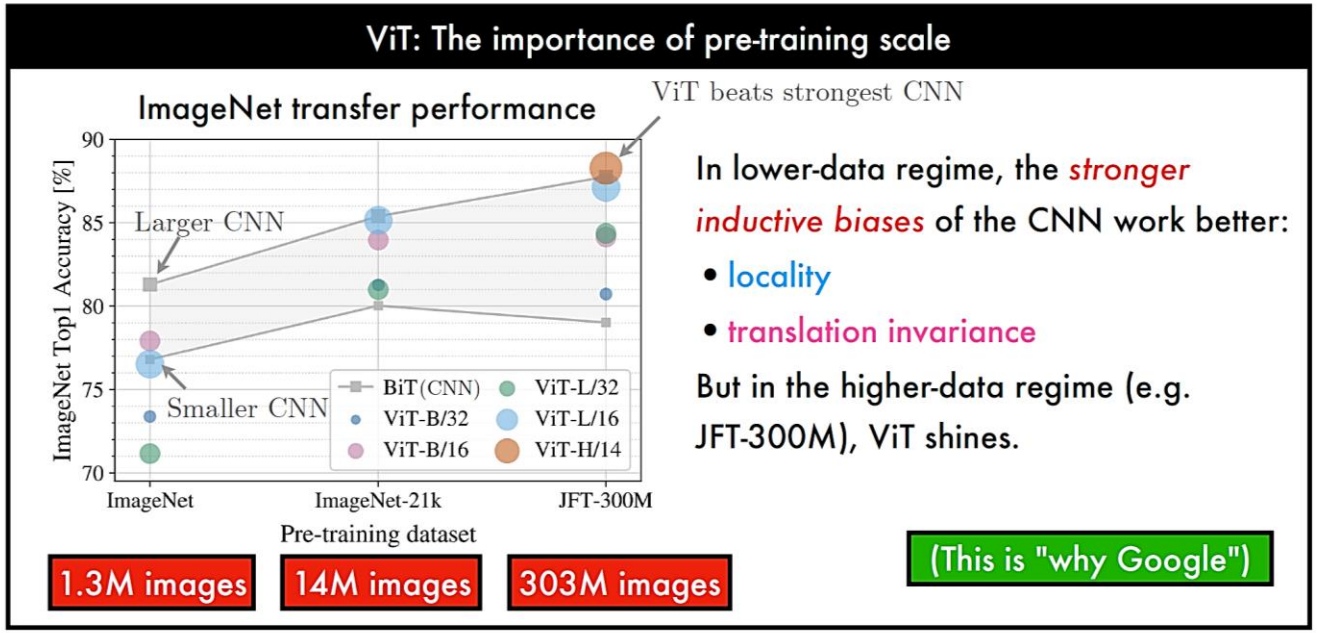
Transformer vs CNN

- **Similarities:**
  - Both detect **patterns/similarities** across inputs.
  - CNN filters slide over an image to detect **local features** (edges, textures, shapes).
  - Attention heads compare embeddings (via dot products) to detect **relationships** between tokens/patches.
  - Both feed their outputs into later layers (e.g., fully connected layers) to build higher-level representations.

- **Differences:**
  - **Local vs. global:** CNNs detect features in fixed local regions; attention considers **all pairs** of inputs, allowing global context.
  - **Static vs. dynamic:** CNN filters are fixed once learned; attention weights are **dynamic**, changing depending on the input.
  - **Inductive bias:** CNNs assume spatial locality (good for images); Transformers are more flexible but need more data to learn structure.
  - **Computation:** CNNs are efficient for local patterns; attention scales quadratically with sequence length.

Scaling Vision Transformer

- CNNs have strong inductive biases (built-in assumptions like locality and translation invariance) → work well in low-data regimes (e.g., ImageNet with 1.3M images)
- Transformers (ViTs) have little prior bias → need much more data to learn structure
- With huge datasets (e.g., JFT-300M), ViTs surpass the best CNNs
- Key insight : More data reduces the need for strong inductive biases
- That's why Google was the first to make ViTs work — they had the scale (hundreds of millions of images)



CNN – Transformers summary

The Evolution:

- CNNs excel with limited data through spatial inductive biases (locality, translation invariance)
- RNNs introduced sequential processing but suffered from vanishing gradients and slow computation
- Transformers revolutionized AI through parallel attention mechanisms enabling global context

Vision Transformers :

- Small datasets (<10M): CNNs outperform due to built-in assumptions about spatial structure
- Large datasets (300M+): ViTs surpass CNNs by learning both local patterns AND global relationships
- Data scale reduces the need for hand-crafted inductive biases

Self-Attention:

- Embeddings "communicate" through Query-Key-Value projections
- Dynamic, input-dependent weighting (not fixed like CNN filters)
- Multi-head attention captures diverse relationships simultaneously

- Transformers now power: image classification, segmentation, detection, captioning, and multimodal models (CLIP, GPT-4V, DALL-E)

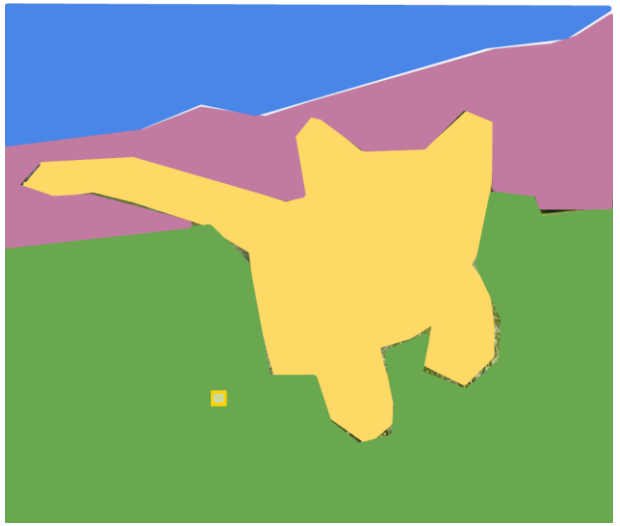
Applications

Image Classification



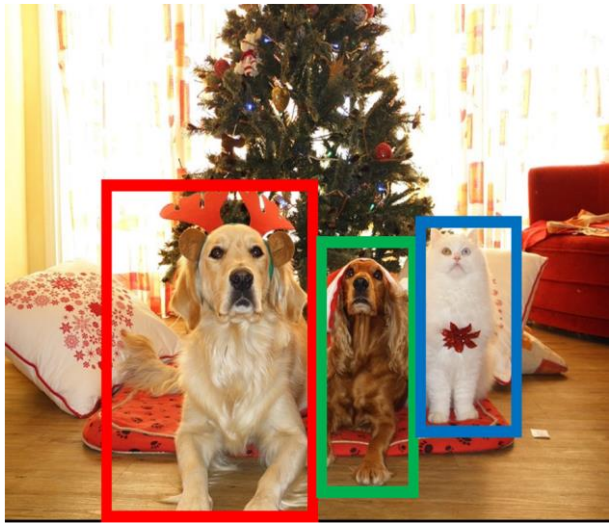
CAT

Semantic Segmentation



GRASS, CAT, TREE, SKY

Object detection



DOG, DOG, CAT

Instance segmentation



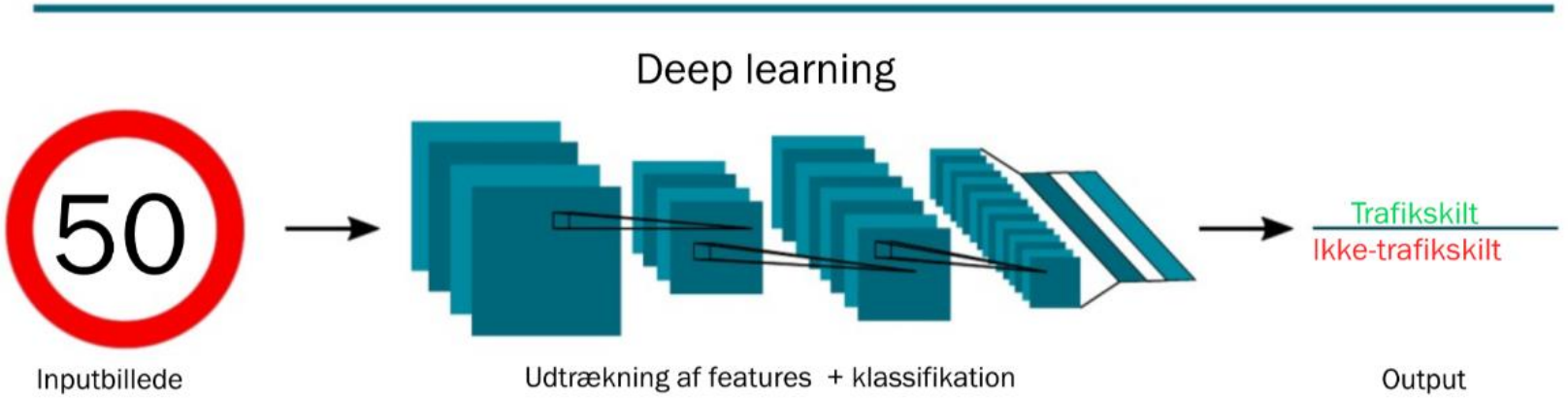
DOG, DOG, CAT

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Applications

So if no human knowledge is involved in learning the features – how is it done?

By looking at LOTS of example data!



Datasets



Pascal VOC 2005
4 classes
1578 images
➔ 2012:
20 classes
11k images



MS COCO 2015
330k images
80 classes



LVIS 2019
160k images, 1203 classes, 2M instance annotations



SA-1B 2023
11M images, 1B+ masks

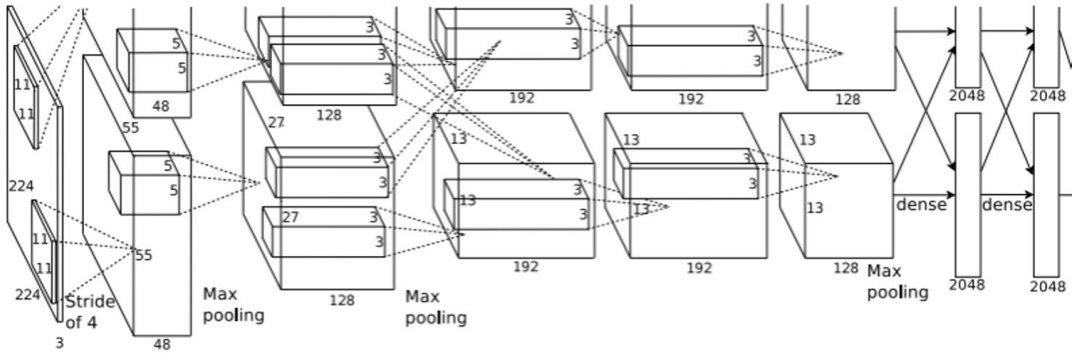
SA-V 2024
51k videos
643k masklets

(no classes)

Image classification

Image classification





Fully-connected
4096 to 1000

Class scores
Cat: 0.9
Dog: 0.05
Car: 0.01
…

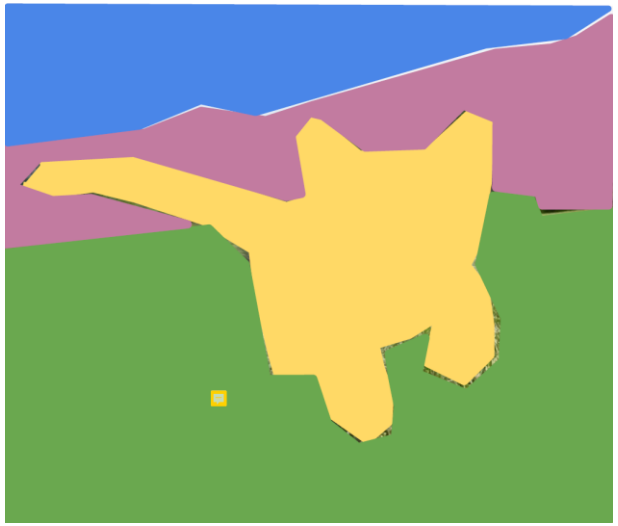
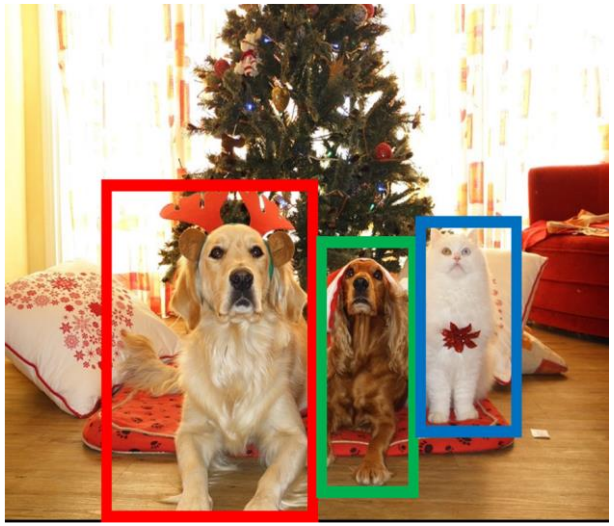
AlexNet (picture)
VGG
ResNet
…

Vector:
4096

AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Computer Vision Tasks

Classification	Semantic Segmentation	Object detection	Instance segmentation
			
CAT	GRASS, CAT, TREE, SKY	DOG, DOG, CAT	DOG, DOG, CAT
No spatial extent	No objects, just pixels	Multiple objects	

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

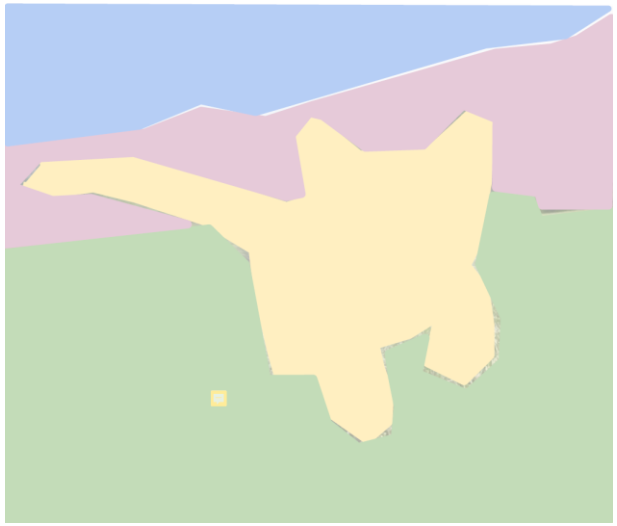
First: Object detection

Classification



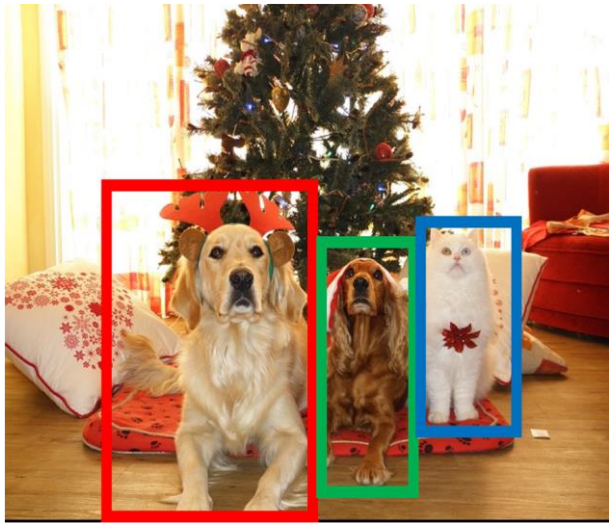
CAT

Semantic Segmentation



GRASS, CAT, TREE, SKY

Object detection



DOG, DOG, CAT

Instance segmentation



DOG, DOG, CAT

No spatial extent

No objects, just pixels

Multiple objects

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Object detection

Object detection: Task definition

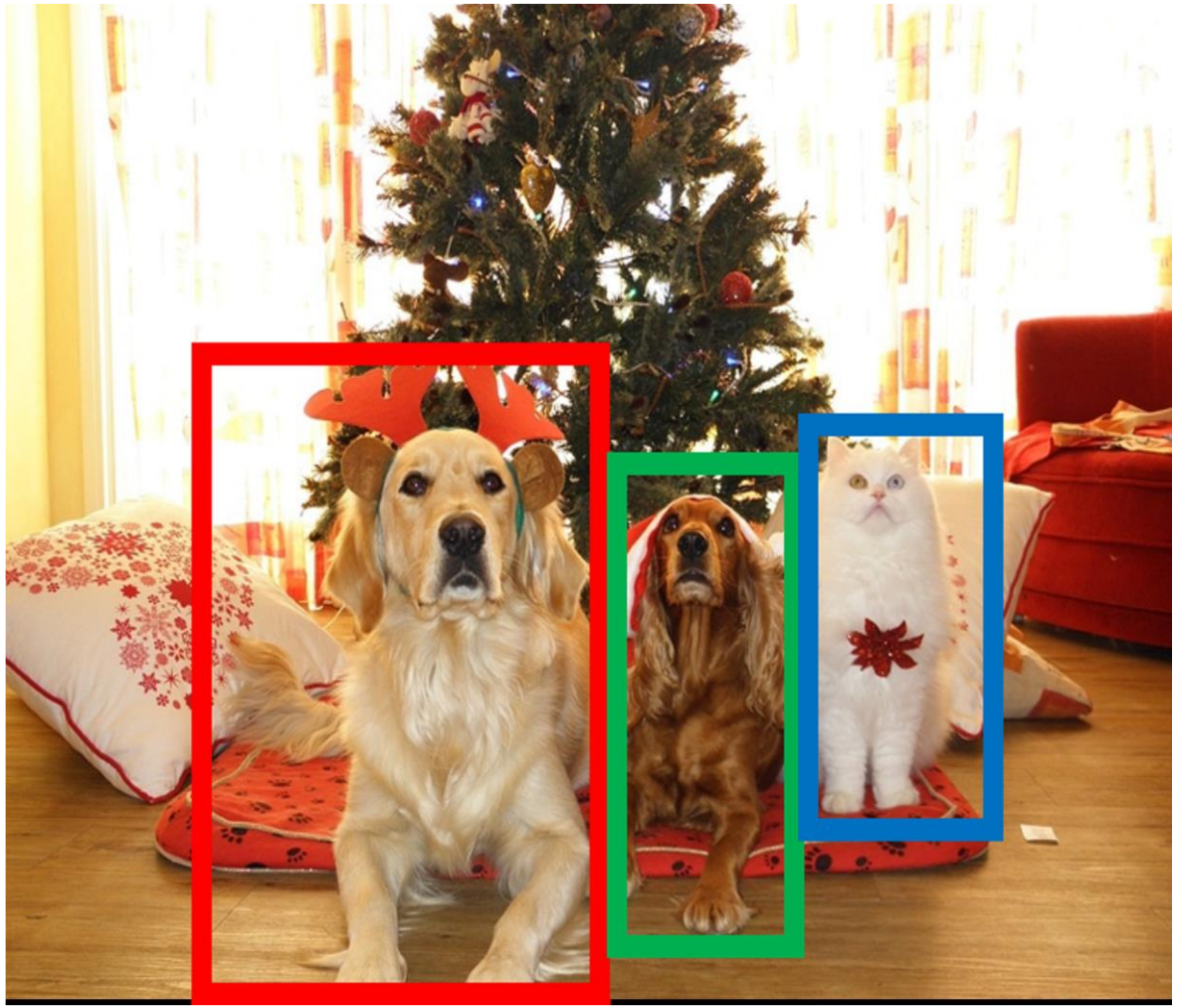
- Input: Single RGB Image



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Object detection: Task definition

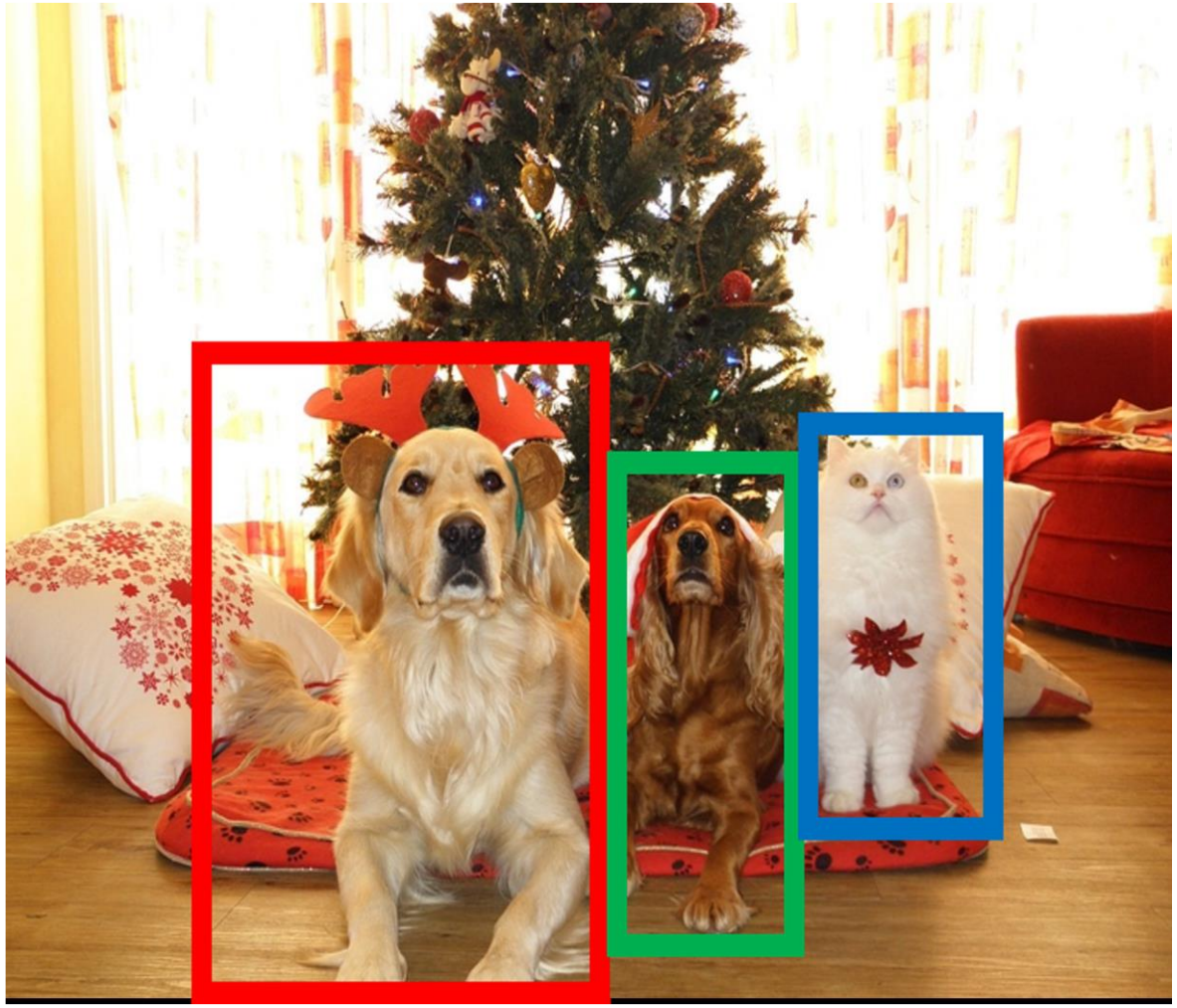
- Input: Single RGB Image
- Output: A *set* of detected objects.
  For each predicted object:
  - Category label (class)
    From fixed set of categories
  - Bounding box
    x, y, width, height



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Object detection: Challenges

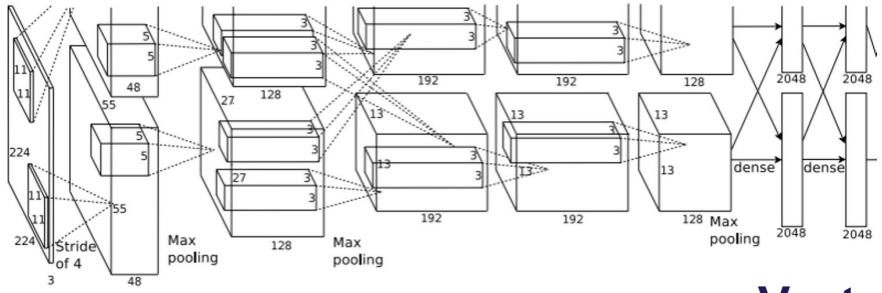
- **Multiple outputs:**
  Need to output variable number of objects per image
- **Multiple types of output:**
  Need to predict ”what” (class) as well as “where” (bounding box)
- **Large images:**
  Classification works at 224x224; need higher resolution for detection, at least 640x640



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting a single object





"What"

Fully Connected: 4096 to 1000

Class scores
Cat: 0.9
Dog: 0.05
Car: 0.01
…

Correct label:
Cat

Softmax loss

Multitask loss

Weighted sum

Loss

Vector: 4096

Fully Connected: 4096 to 4

Box coordinates
(x, y, w, h)

L2 loss

Correct box:
(x', y', w', h')

"Where"

What: Classification
Where: Regression

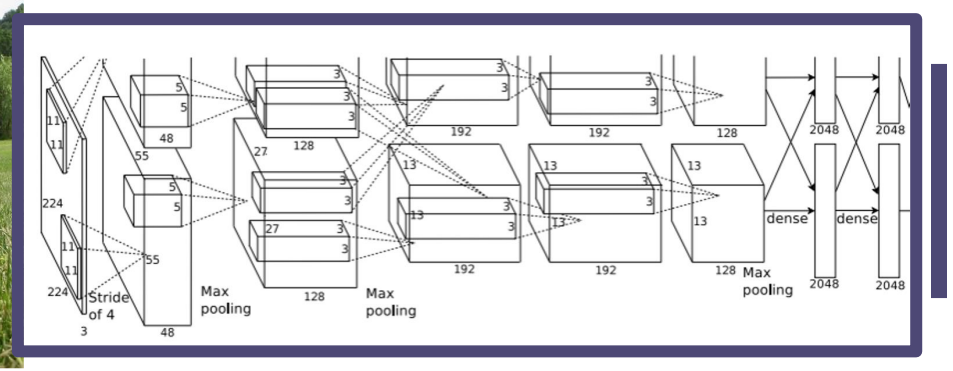
AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting a single object



Often pretrained on ImageNet (transfer learning)



- Only works with 1 object per image!
- We need to detect multiple objects per image

”What”

Fully Connected: 4096 to 1000

Class scores
Cat: 0.9
Dog: 0.05
Car: 0.01
…

Softmax loss

Correct label:
Cat

Multitask loss

Weighted sum → **Loss**

Vector: 4096

Fully Connected: 4096 to 4

Box coordinates (x, y, w, h)

L2 loss

Correct box: (x’, y’, w’, h’)

”Where”

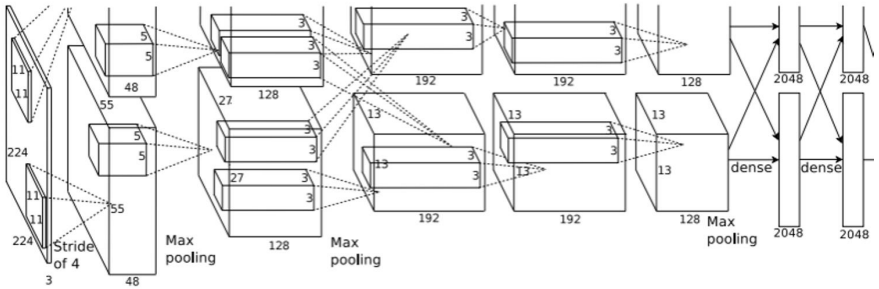
AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects

Need variable number of outputs per image

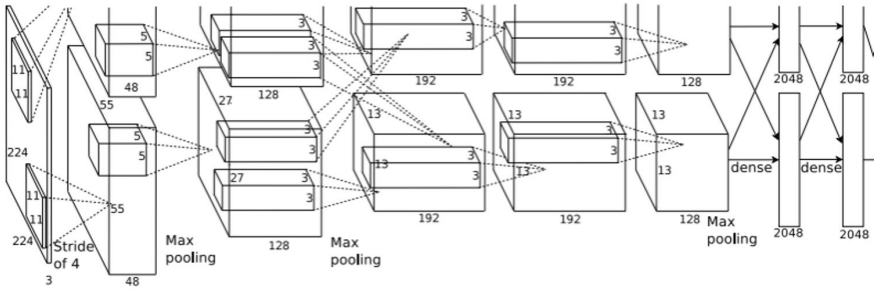




CAT: (x, y, w, h)

4 numbers

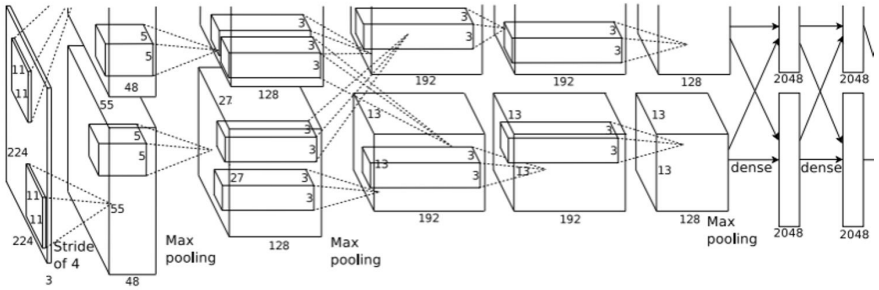




DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers





DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

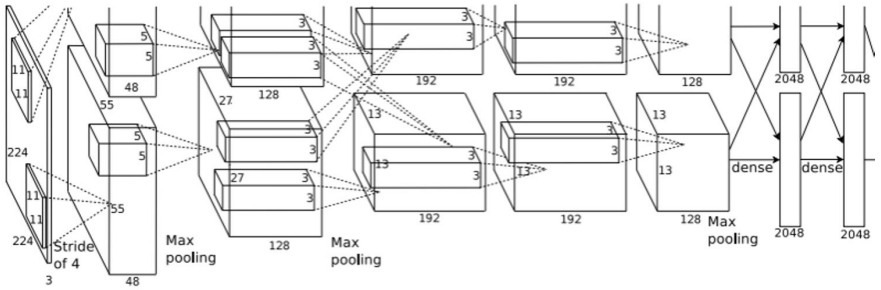
k x 4 numbers

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



Dog? No
Cat? No
Car? No
Background? Yes

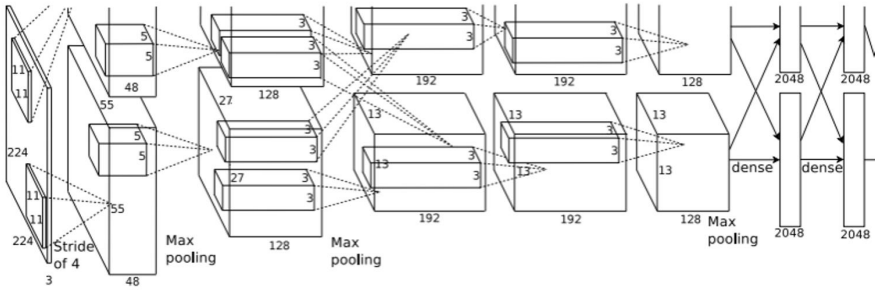
AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



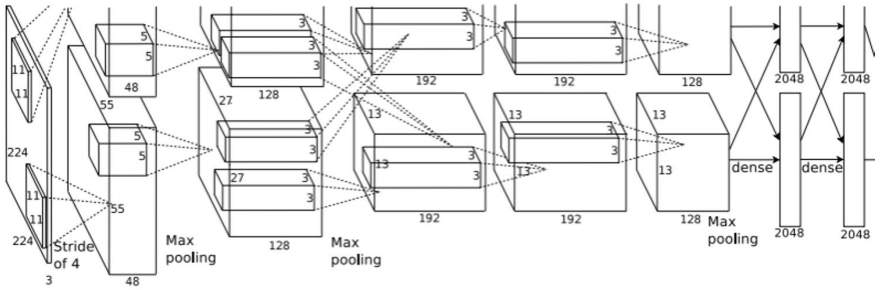
Dog? Yes
Cat? No
Car? No
Background? No

AALBORG UNIVERSITY
DENMARK

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



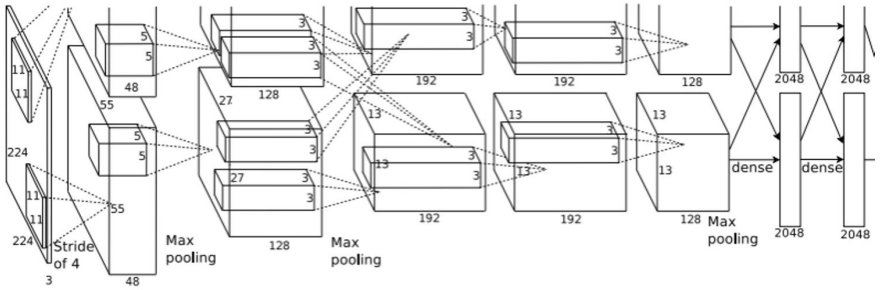
Dog? Yes
Cat? No
Car? No
Background? No

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- For each window position:
  - Warp image, apply a CNN
  - Classification problem (with extra background category)



Dog? No
Cat? Yes
Car? No
Background? No

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Sliding window



- How many boxes to evaluate?
- Parameters
  - Image width (W), height (H)
  - Box width (w), height (h)

- 800 x 600 image
  ~ 58M boxes!

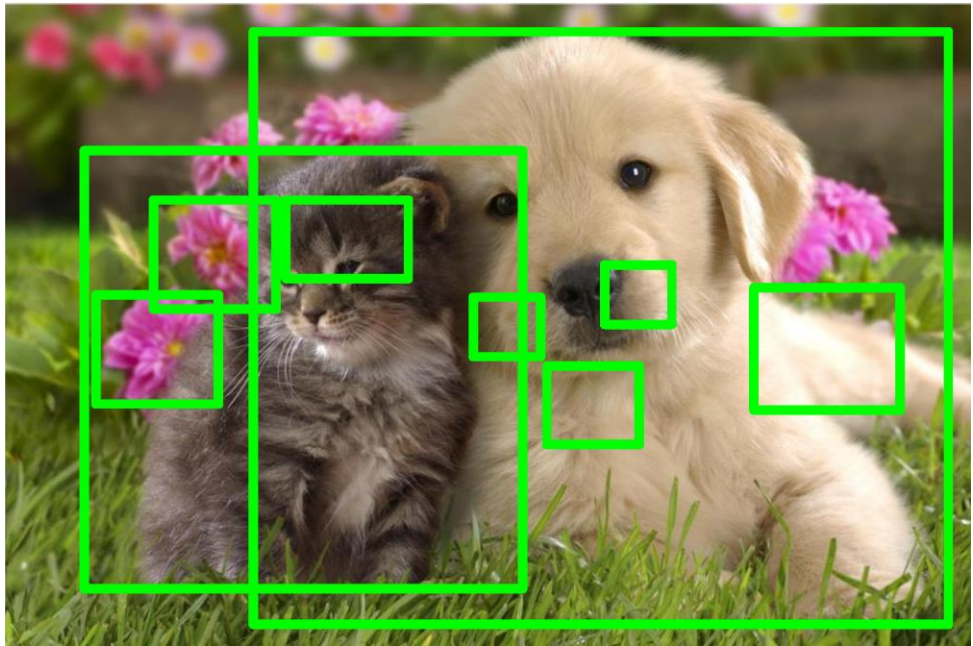
- Infeasible to evaluate all box positions and sizes

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Detecting multiple objects: Region proposals

- Find a small set of boxes that are likely to cover all objects
- Often based on heuristics
  - Look for “blob-like” image regions
- Relatively fast to run
  - **Selective Search** (2013) gives 2000 region proposals in a few seconds on CPU

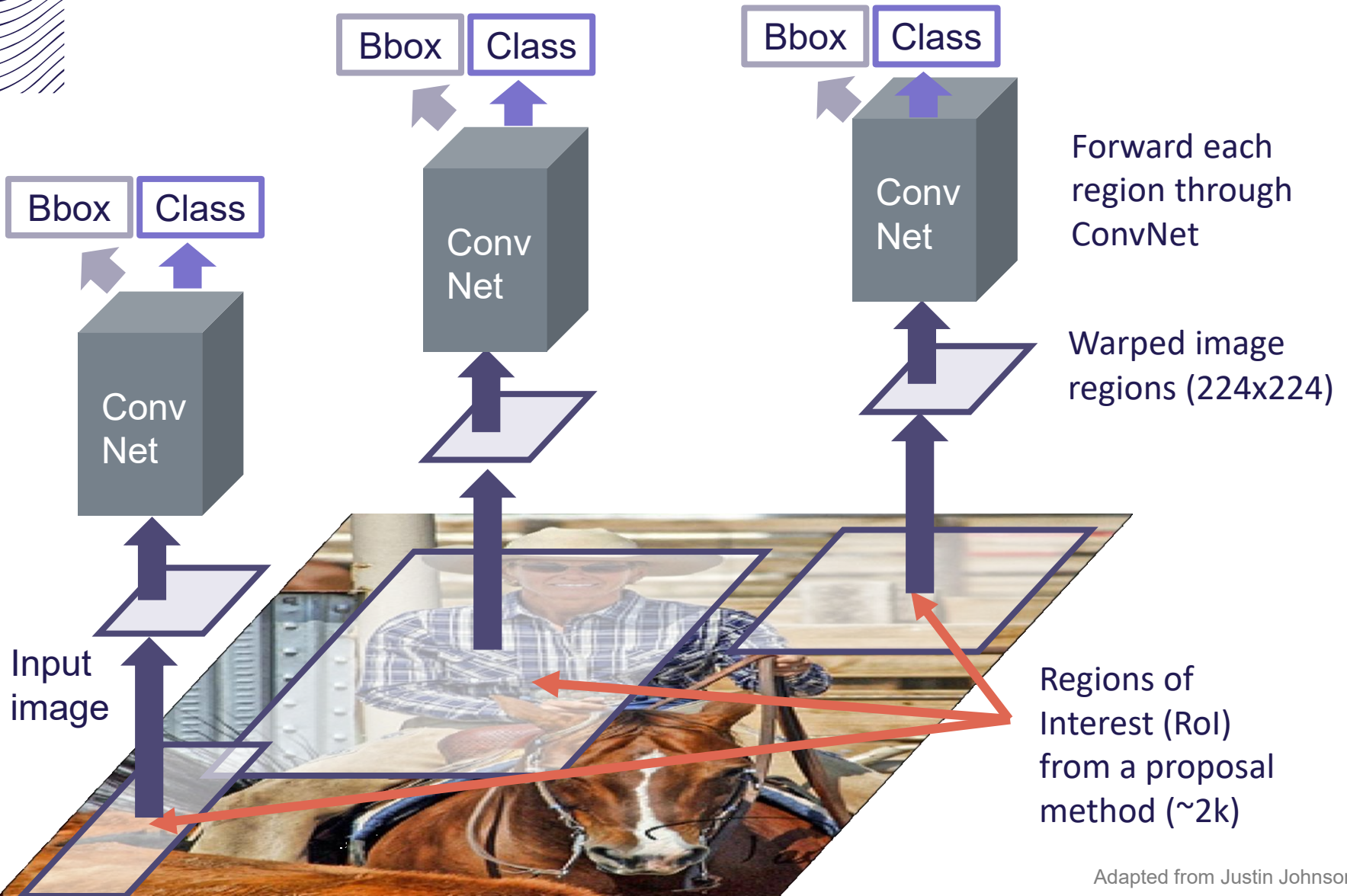




Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013
Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

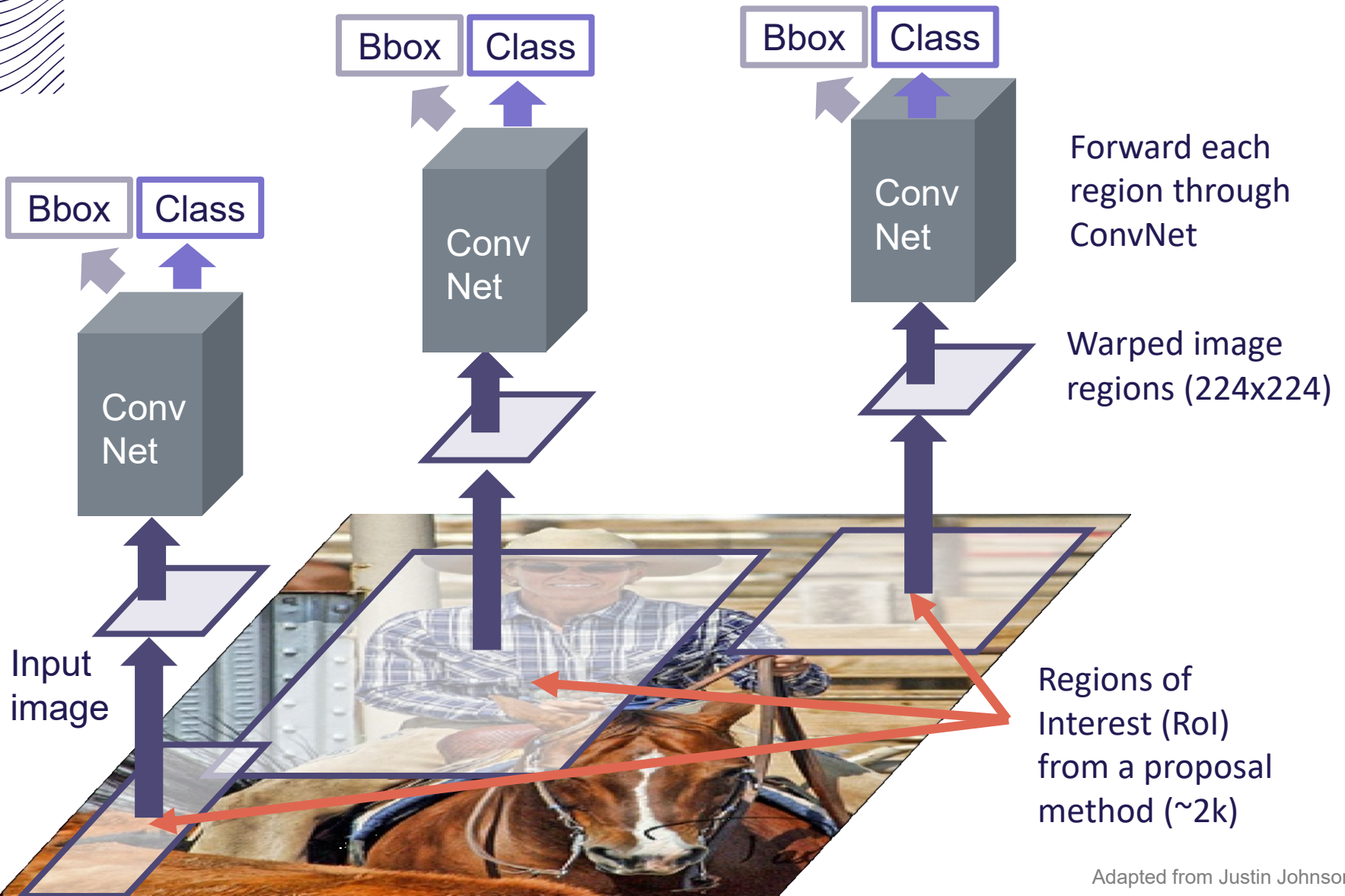
R-CNN: Region-based CNN



Classify each region

Bounding box regression: Predict "transform" to correct the Region-of-interest (ROI). 4 numbers (t_x, t_y, t_h, t_w)

R-CNN: Region-based CNN



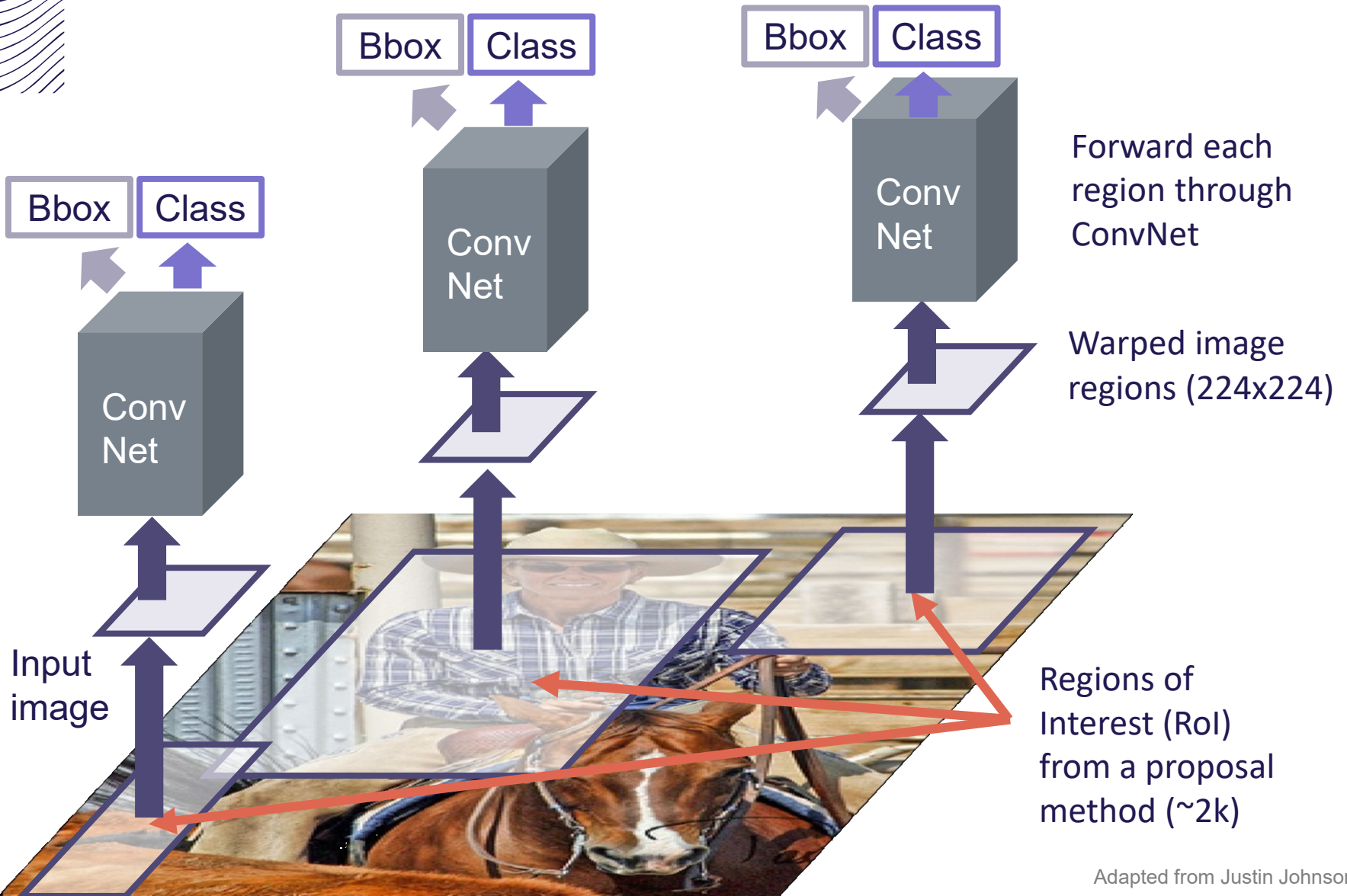
Method-at-a-glance:

For each input RGB image:

1. Run region proposal method (CPU) to compute ~2000 region proposals
2. Resize each region to 224x224 and run independently through CNN to predict class scores, bbox transform
3. Select a subset of region proposals to output
4. Compare with ground-truth-boxes

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

R-CNN: Region-based CNN



Any problems with this approach?

- Separate forward pass for each region proposal
- Very slow!

- Solution: Run CNN only once!

Fast R-CNN

Bbox Bbox Bbox
Class Class Class

Category, box transformation per region

CNN CNN CNN

Small per-region network

Crop + resize features

Regions of interest from proposal method

Image features

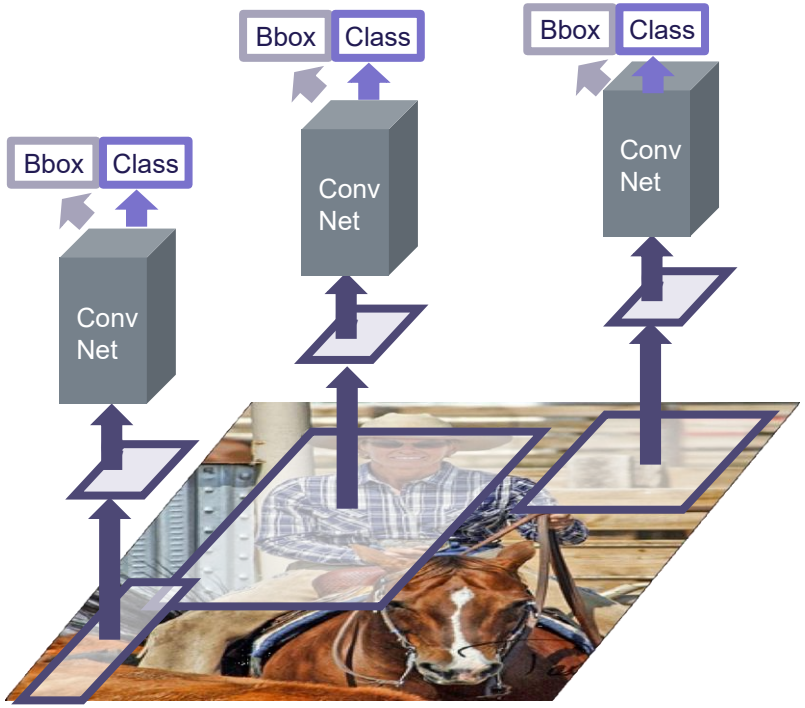
"Backbone network"
- AlexNet
- VGG
- ResNet
- …

ConvNet

Run entire image through ConvNet

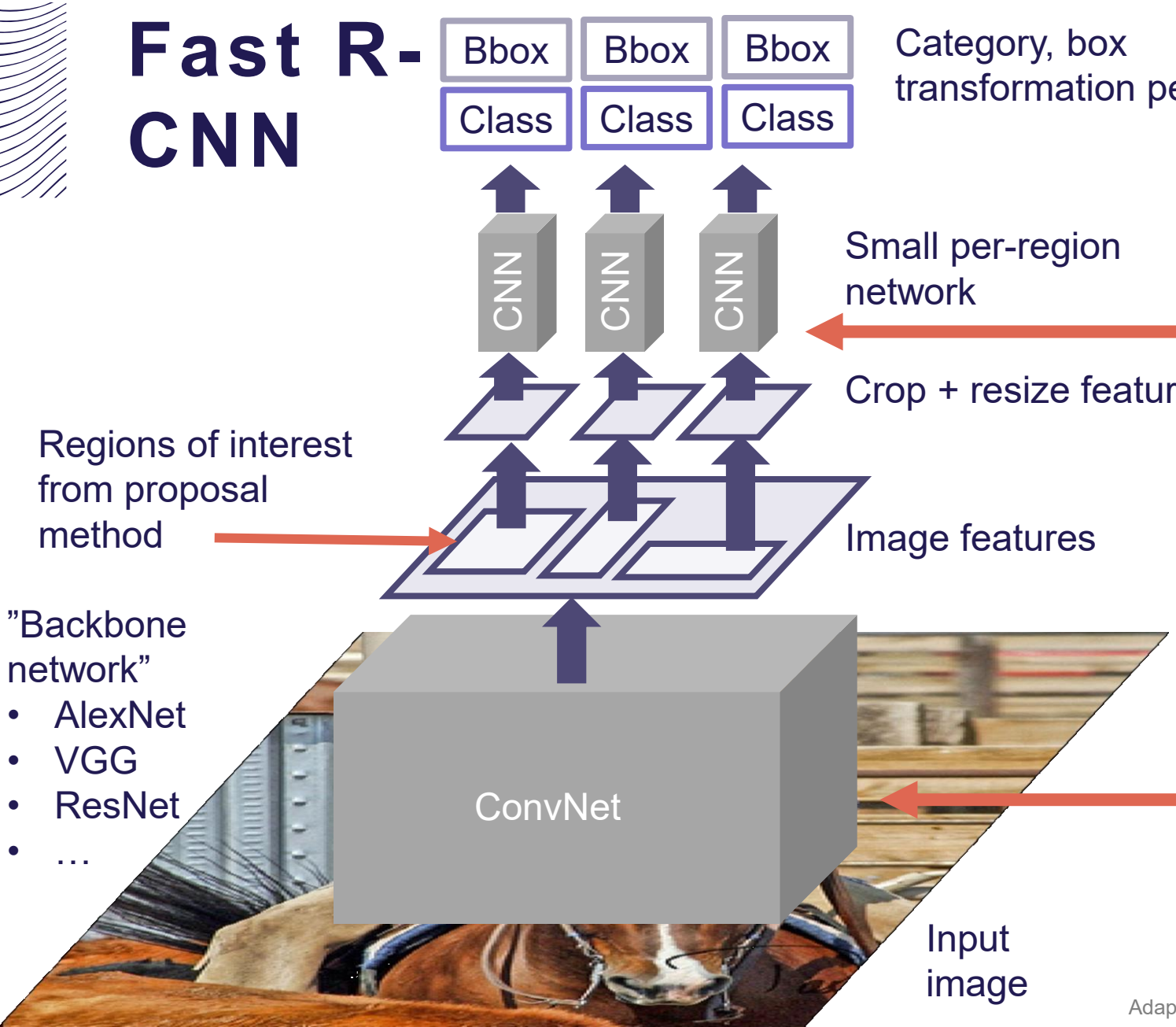
Input image

Original R-CNN



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Fast R-CNN



Category, box transformation per region

Small per-region network

Relatively lightweight per-region network

Crop + resize features

Regions of interest from proposal method

Image features

"Backbone network"

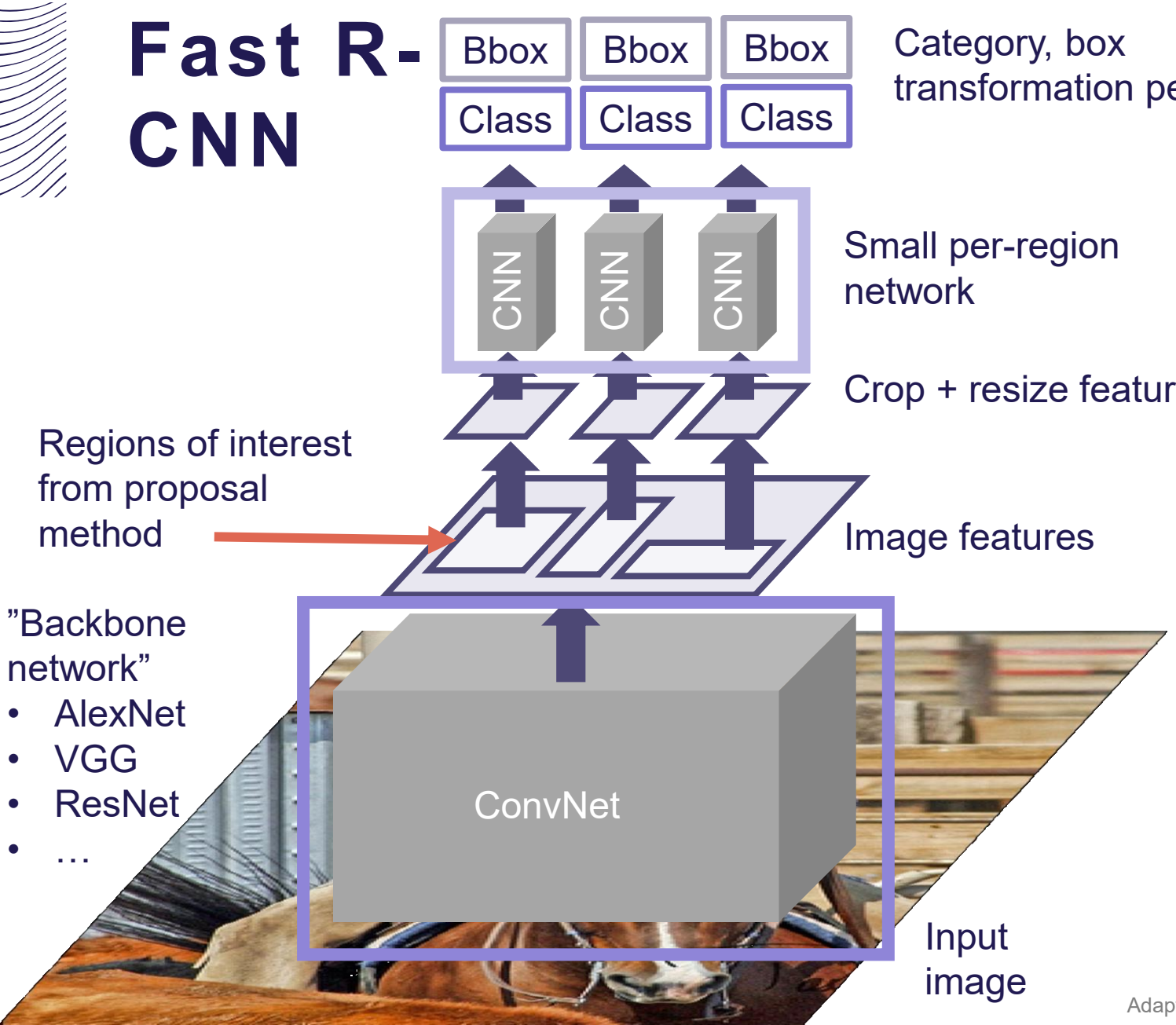
- AlexNet
- VGG
- ResNet
- …

Most of the computation happens in backbone network; once per image

Input image

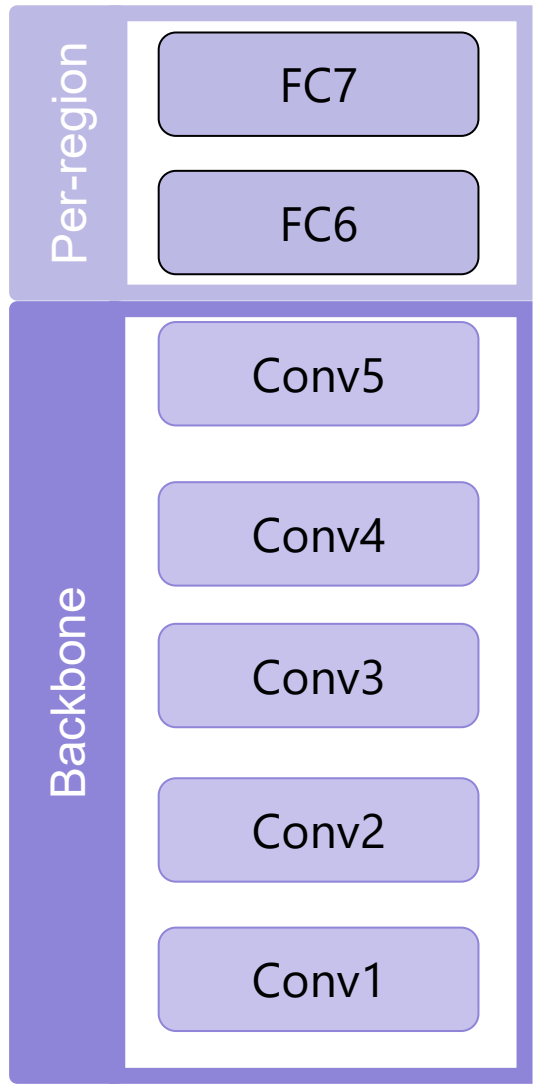
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Fast R-CNN



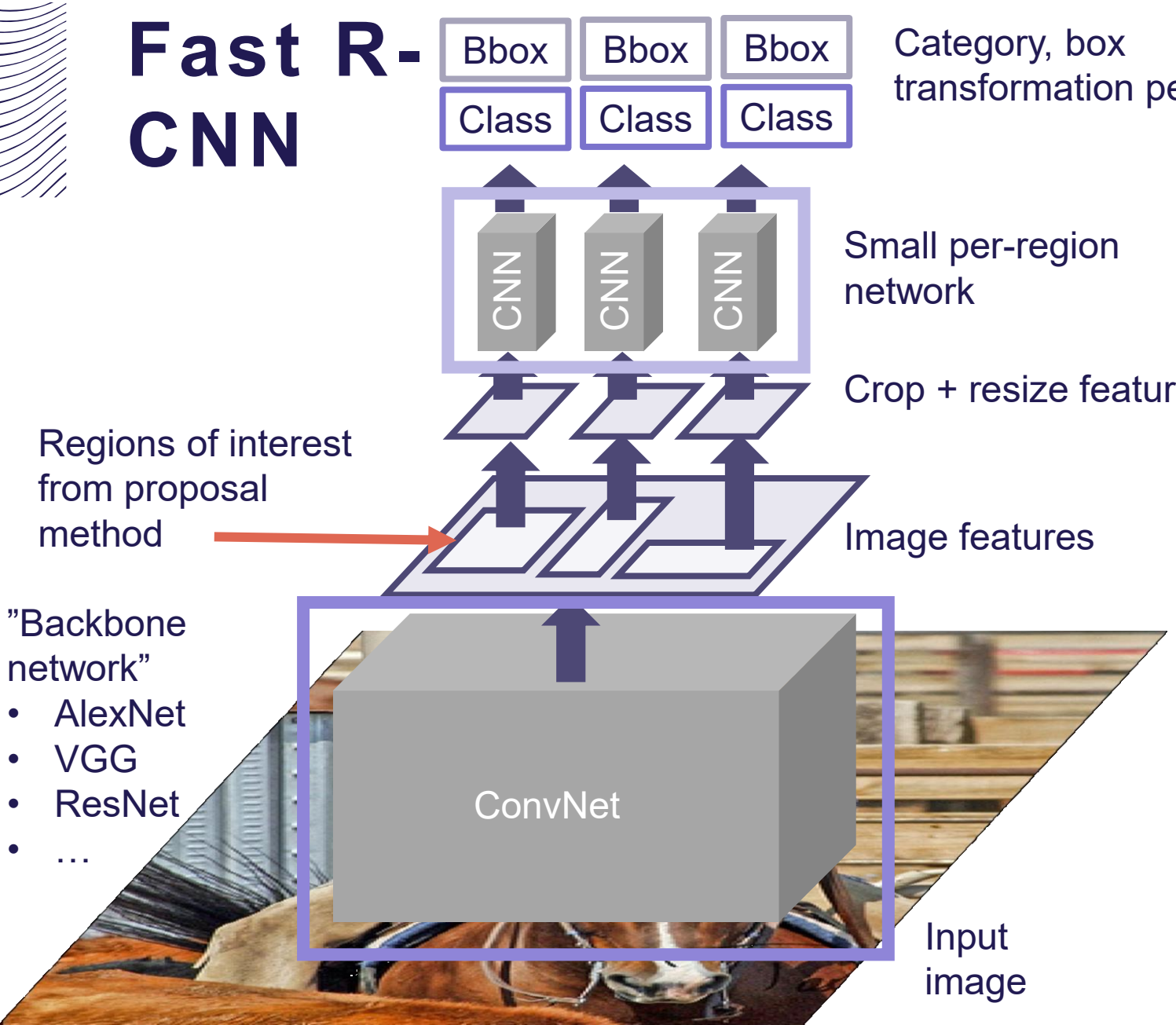
”Backbone network”

- AlexNet
- VGG
- ResNet
- …



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

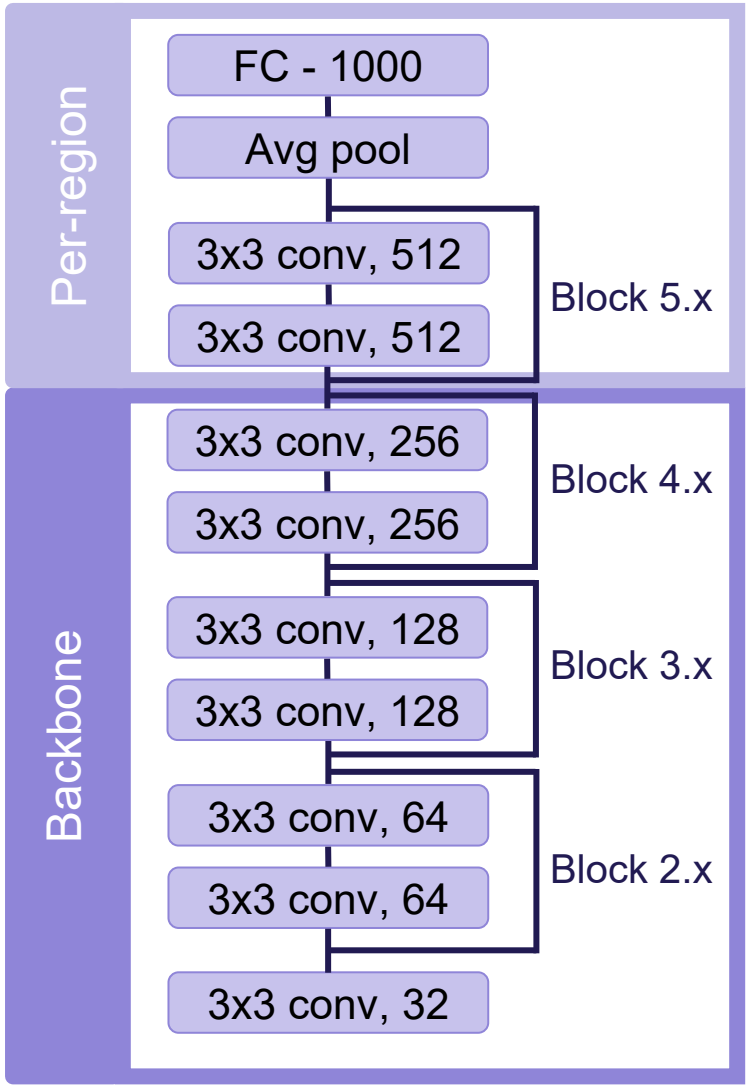
Fast R-CNN



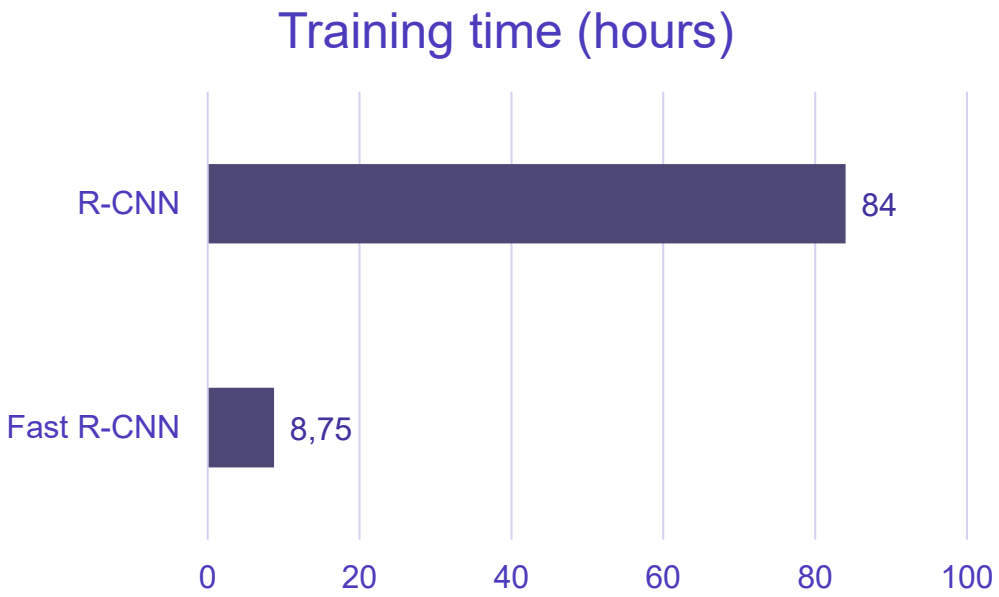
”Backbone network”

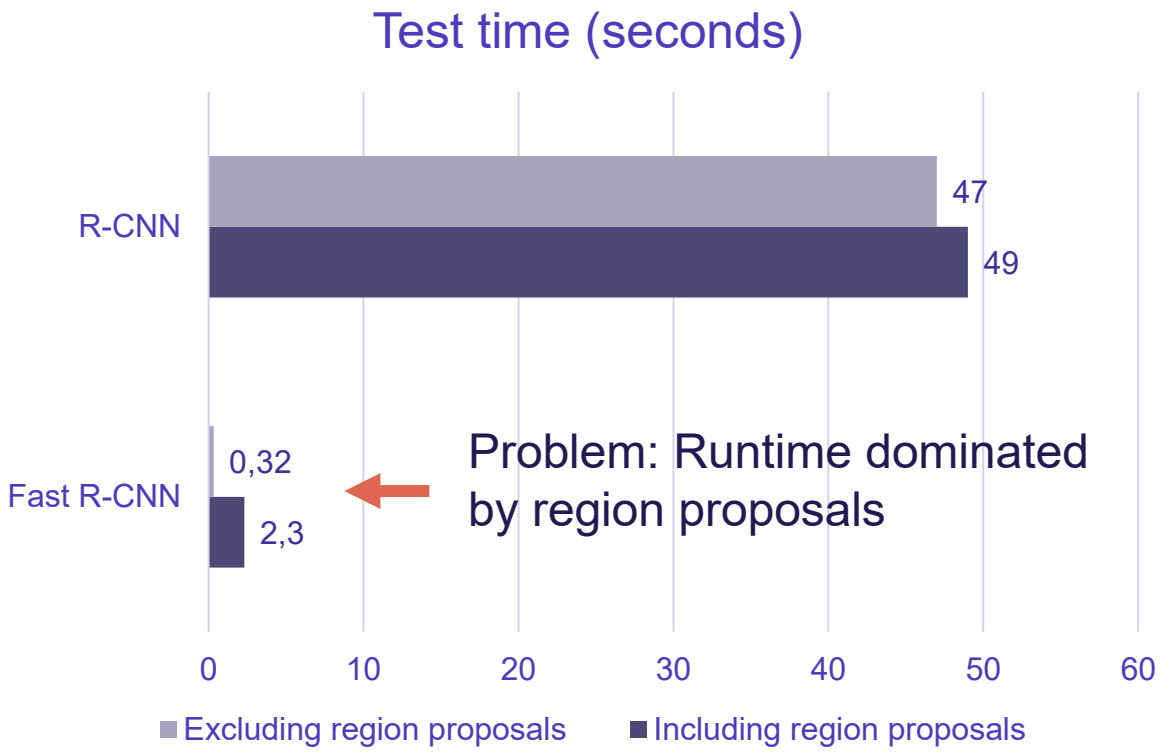
- AlexNet
- VGG
- ResNet
- …

ResNet



Performance



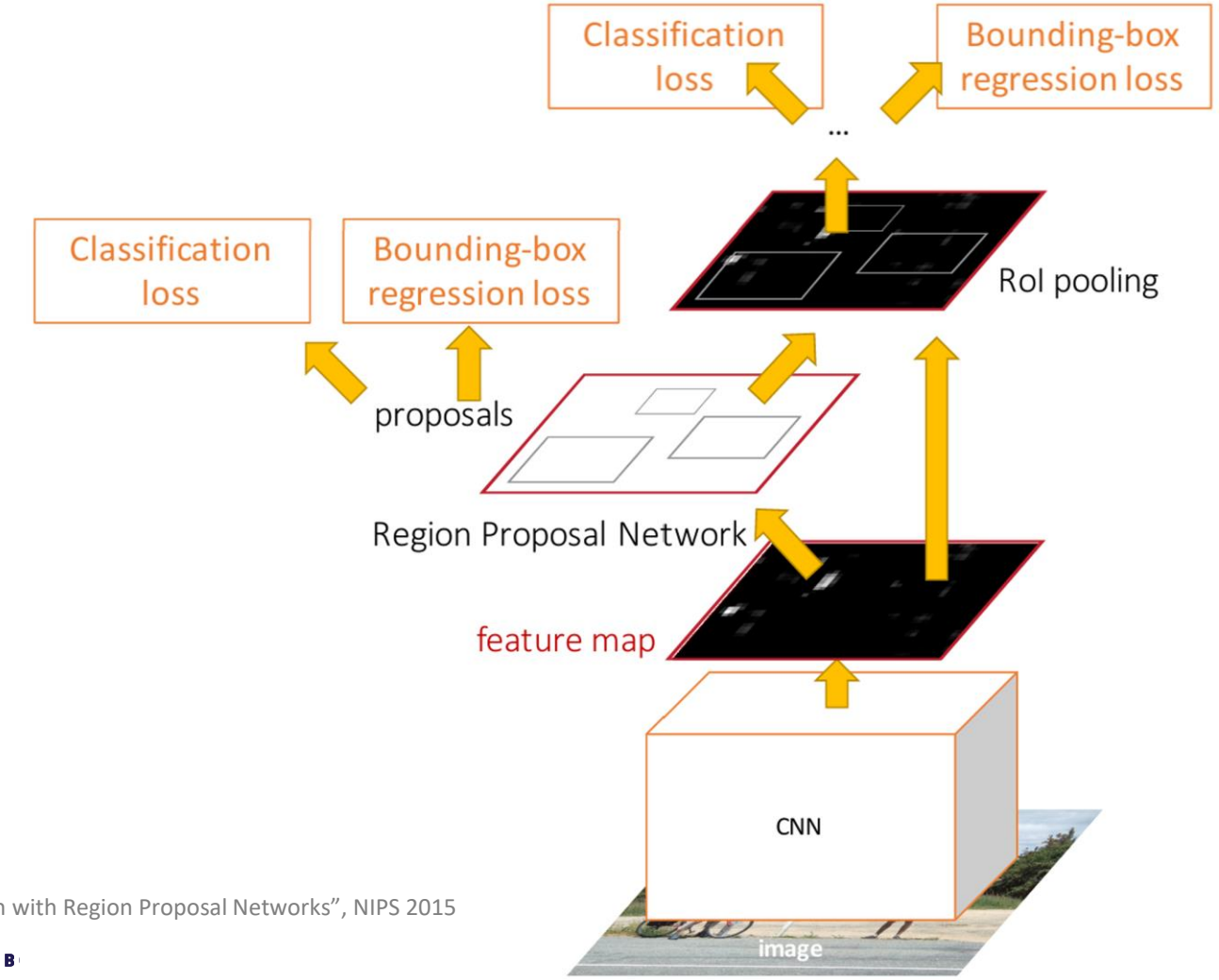


- Region proposals: Computed by ‘Selective Search” algorithm on CPU using classical heuristic methods.
- Let’s replace this with a CNN

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN: Learnable Region Proposals

- Insert Region Proposal Network (RPN) to predict proposals from features
- Otherwise: Same as Fast R-CNN



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

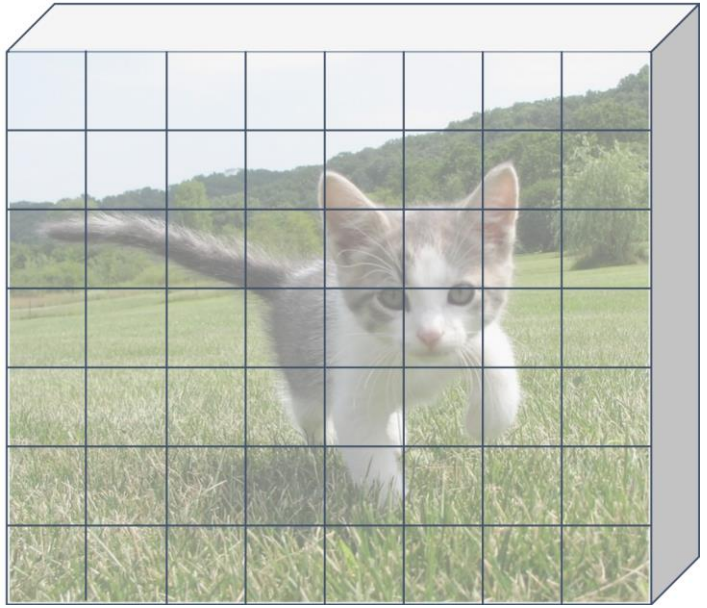
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN Region Proposal Network

Run backbone CNN to get features aligned to input image



CNN



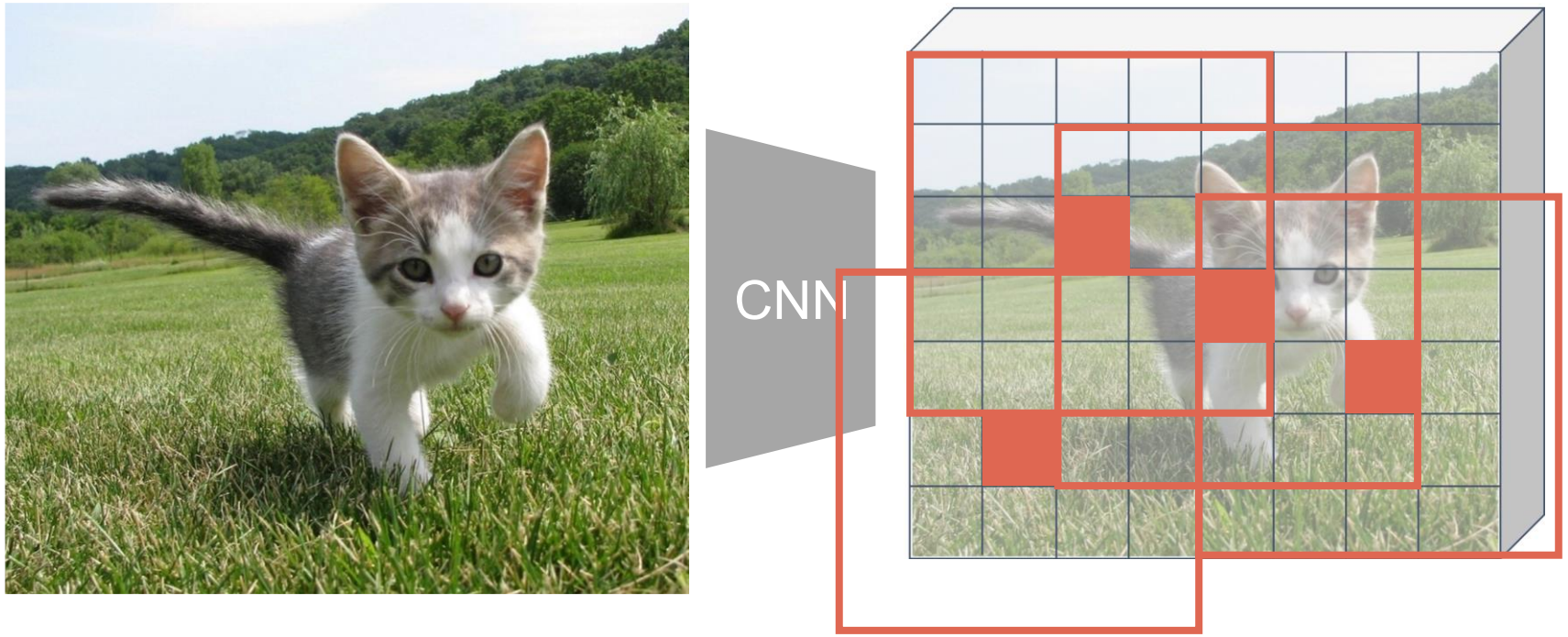
Input image
(e.g. 3 x 640 x 480)

Image features
(e.g. 512 x 20 x 15)

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN Region Proposal Network

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

Image features
(e.g. 512 x 20 x 15)

- Imagine an anchor box of fixed size at each point in the feature map

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN Region Proposal Network

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

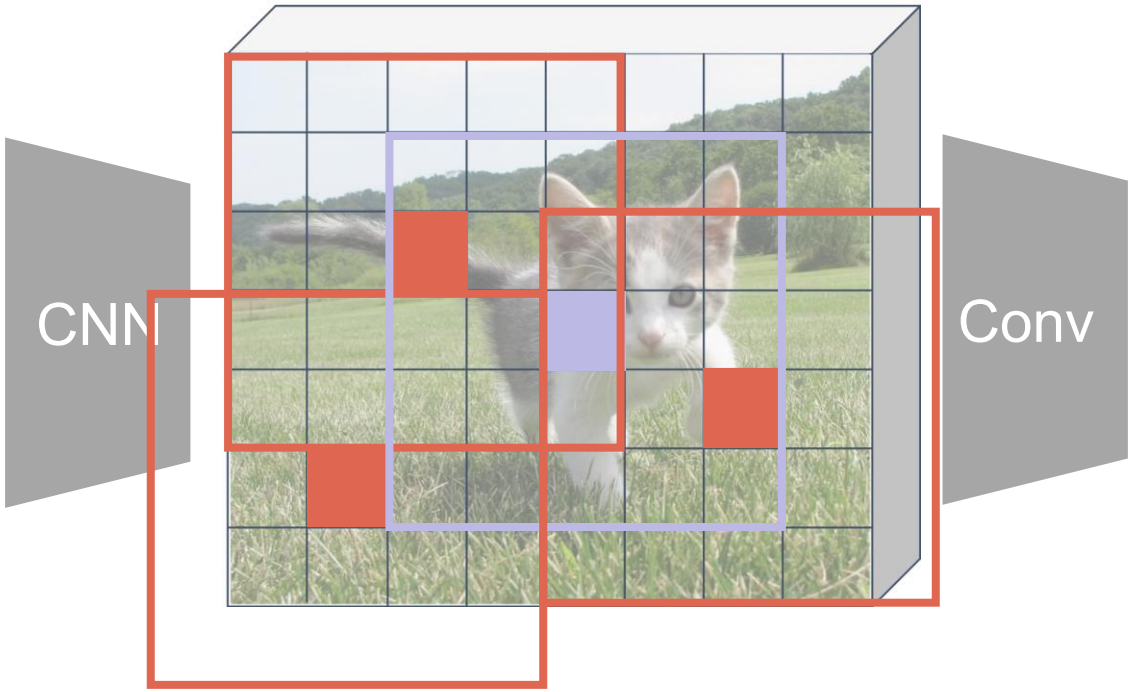


Image features
(e.g. 512 x 20 x 15)

- Imagine an anchor box of fixed size at each point in the feature map

Anchor is an object?
1 x 20 x 15

- At each point, predict whether the corresponding anchor contains an object

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN Region Proposal Network

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

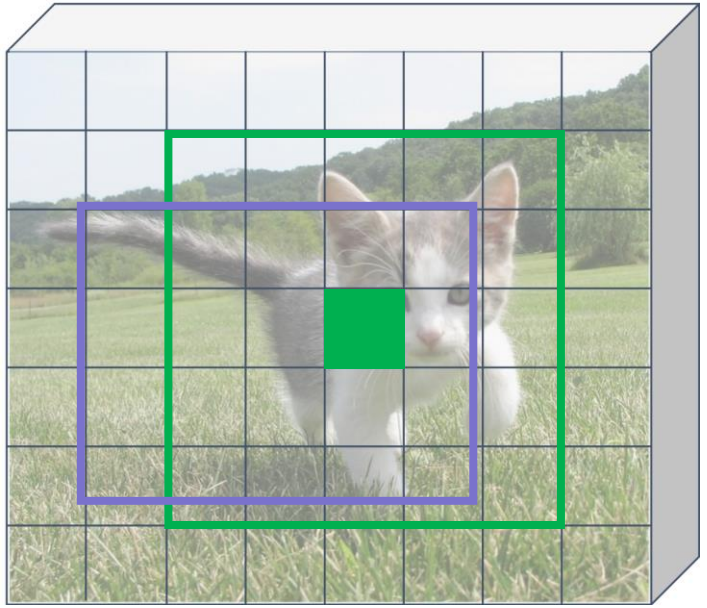


Image features
(e.g. 512 x 20 x 15)

Conv

- Imagine an anchor box of fixed size at each point in the feature map

→ Anchor is an object?
1 x 20 x 15

→ Box transforms
4 x 20 x 15

- For positive boxes, also predict a box transform to regress from anchor box to object box

AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN Region Proposal Network

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

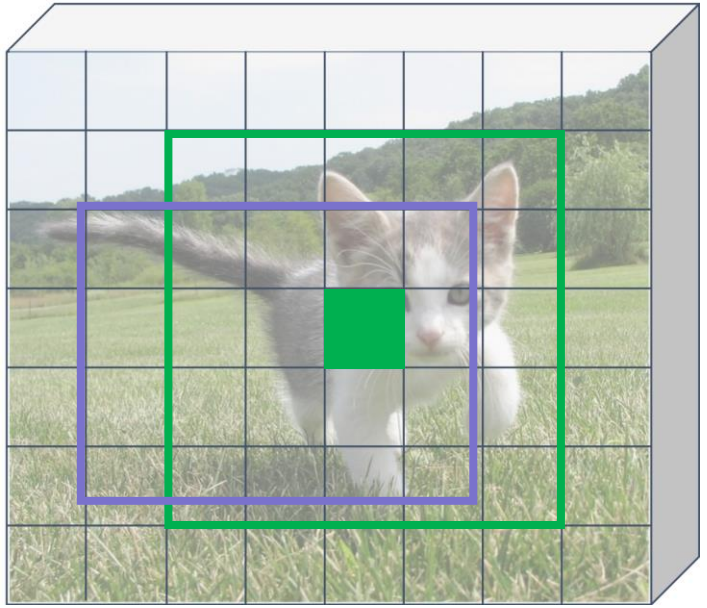


Image features
(e.g. 512 x 20 x 15)

Conv

- Imagine K anchor boxes of fixed size at each point in the feature map

→ Anchor is an object?
K x 20 x 15

→ Box transforms
4K x 20 x 15

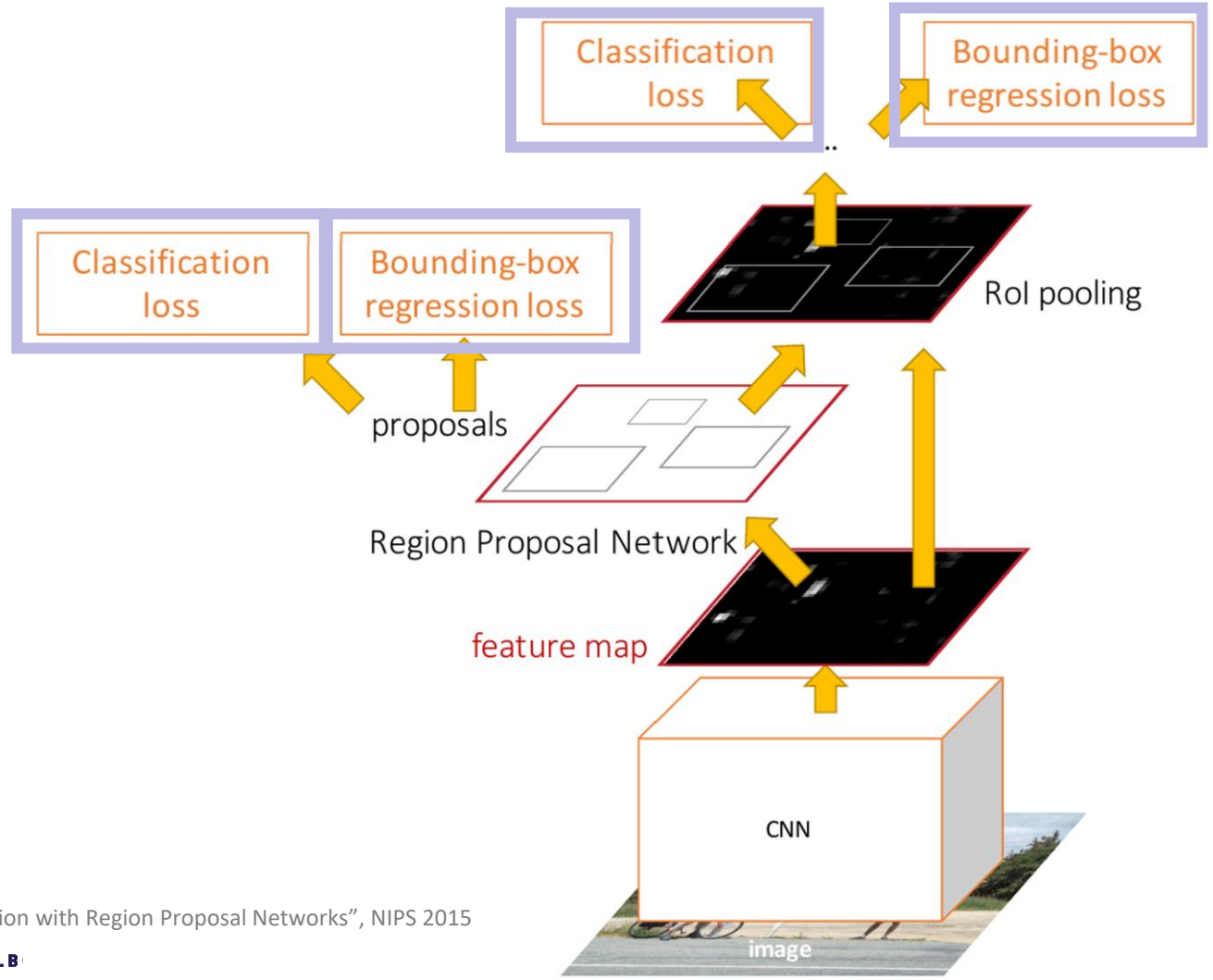
- Use K different anchor boxes at each point to account for variations in size, shape of objects

AALBORG UNIVERSITY
DENMARK

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN: Learnable Region Proposals

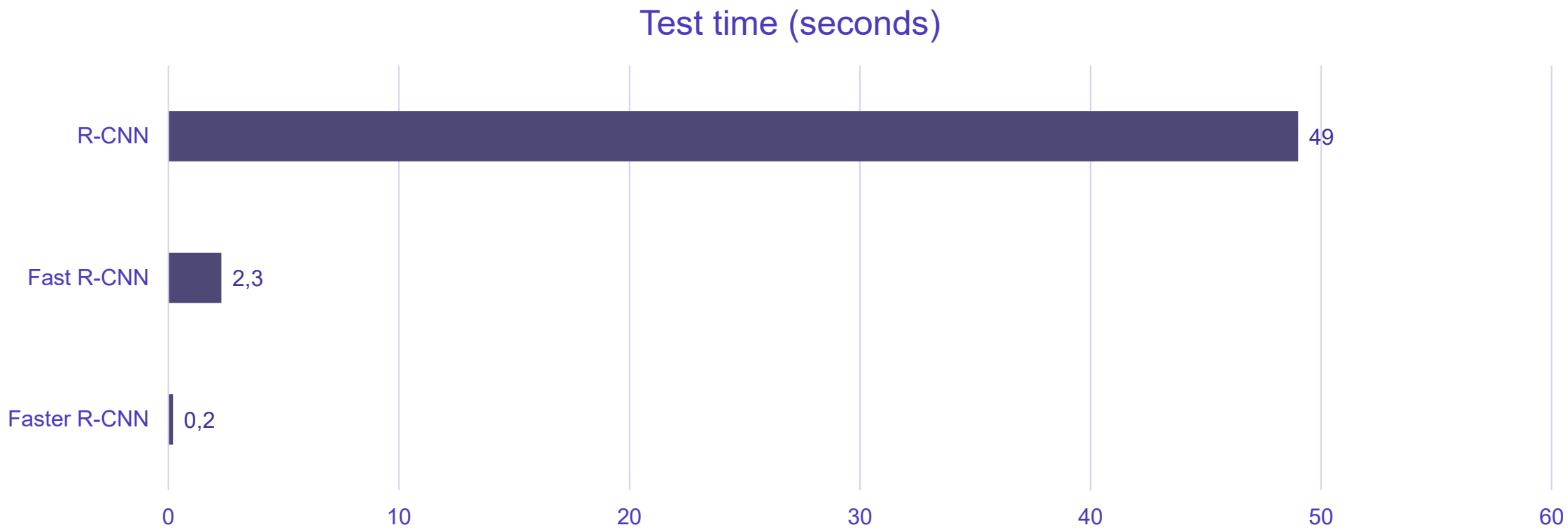
- Joint training phase: (4 losses)
  1. **RPN classification**
     Anchor box is an object?
  2. **RPN regression**
     Predict transform from anchor box to proposal box
  3. **Object classification**
     Classify into background / object classes
  4. **Object regression**
     Predict transform from proposal box to object box



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Improved Performance



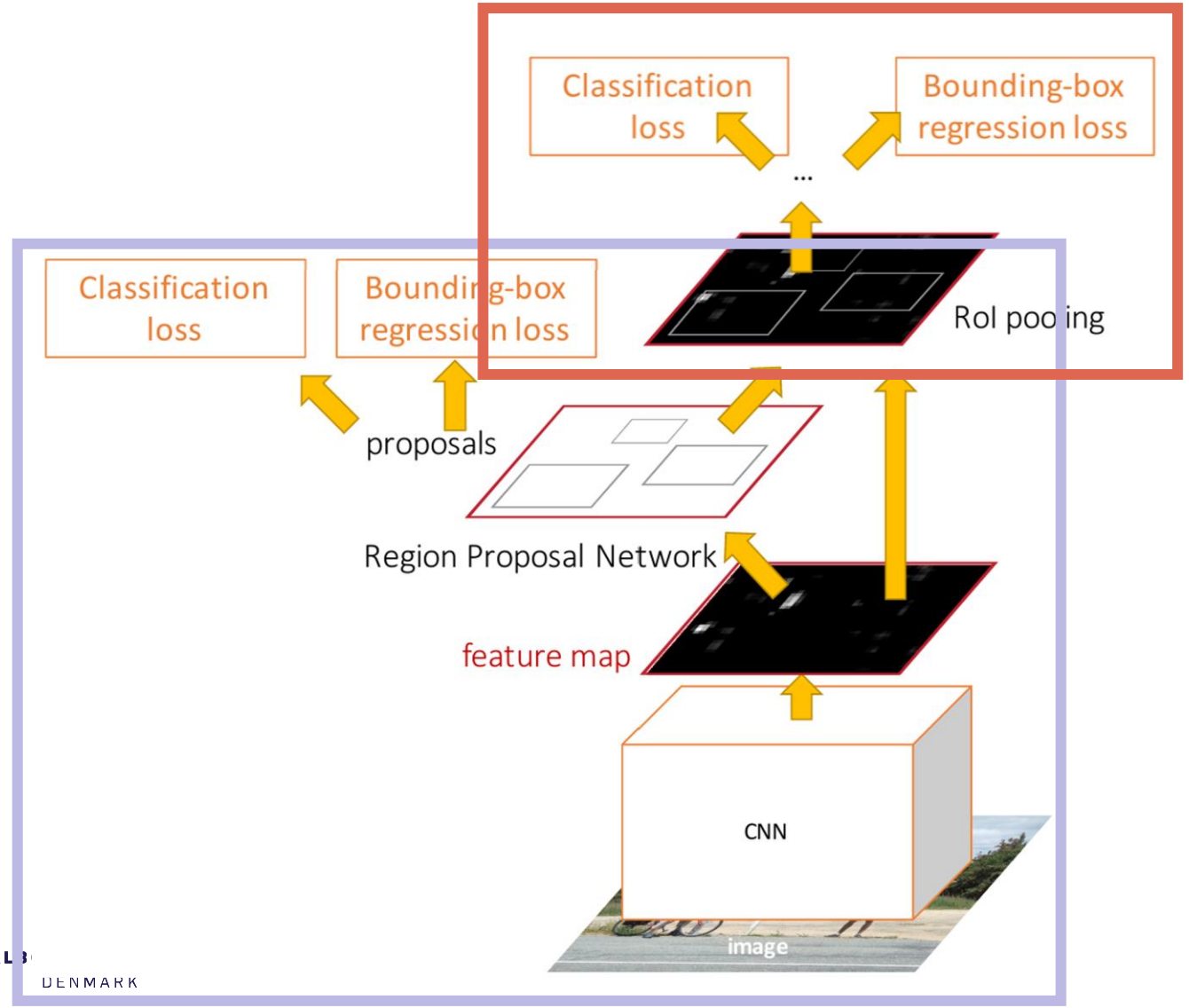
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN: Learnable Region Proposals

- Faster R-CNN is a **two-stage** object detector

- First stage: Run once per image
  - Backbone network
  - Region proposal network

- Second stage: Run once per region
  - Crop features: ROI pool / align
  - Predict object class
  - Predict bounding box offset

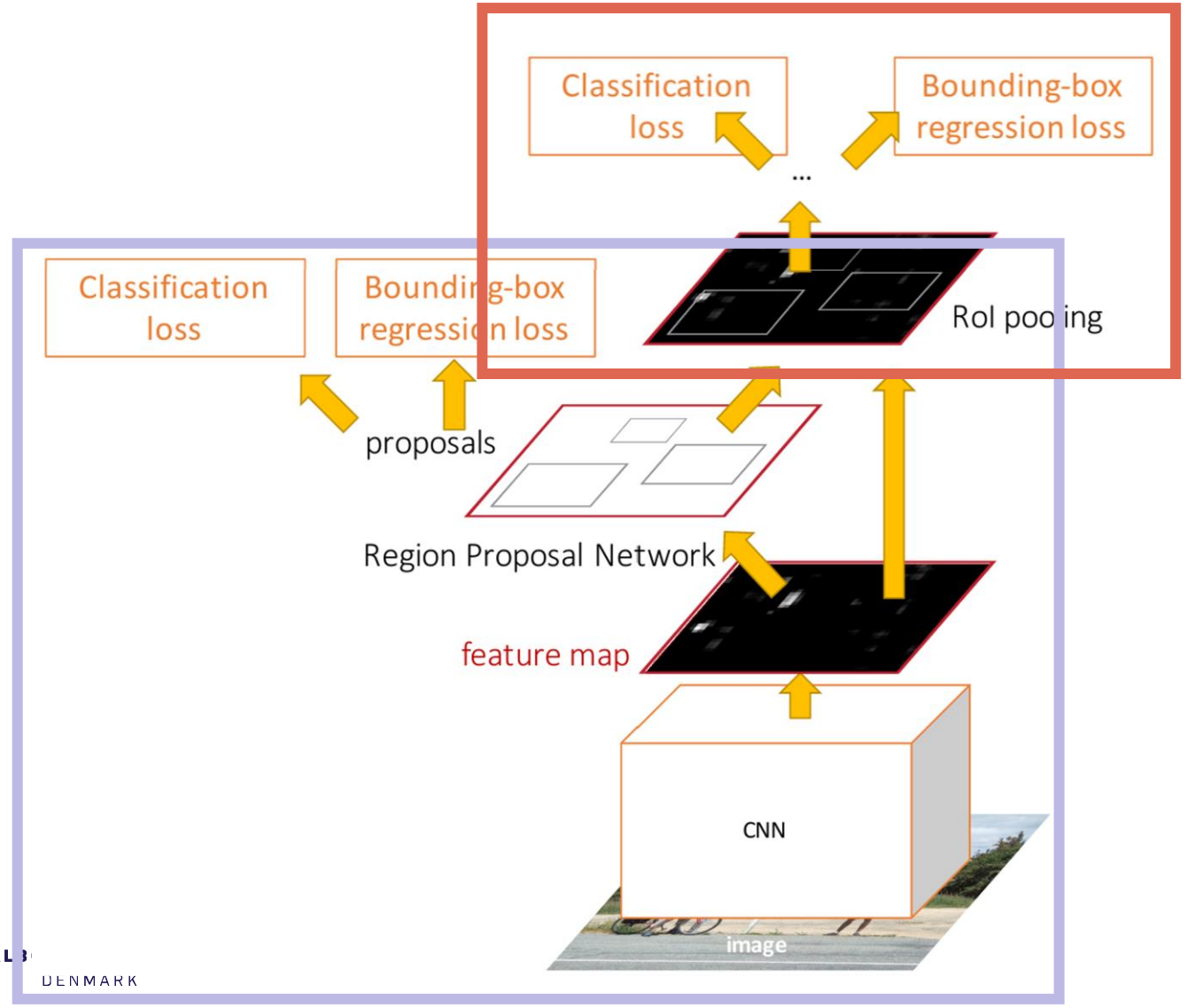


Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Faster R-CNN: Learnable Region Proposals

- Faster R-CNN is a **two-stage** object detector

Why two stages?
Is one stage more efficient?



Single-stage Object Detection

- RPN: Classify each anchor as object / not object

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

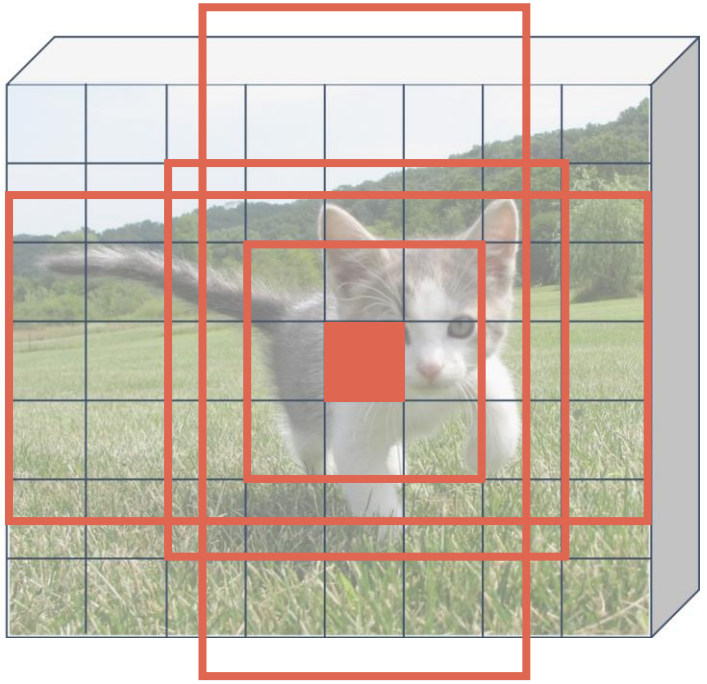


Image features
(e.g. 512 x 20 x 15)

Conv

Anchor category
K x 20 x 15

Box transforms
4K x 20 x 15

- K anchors at each position in image feature map

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Single-stage Object Detection

- RPN: Classify each anchor as object / not object
- Single-stage detector: Classify each object as one of C categories (or background)

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

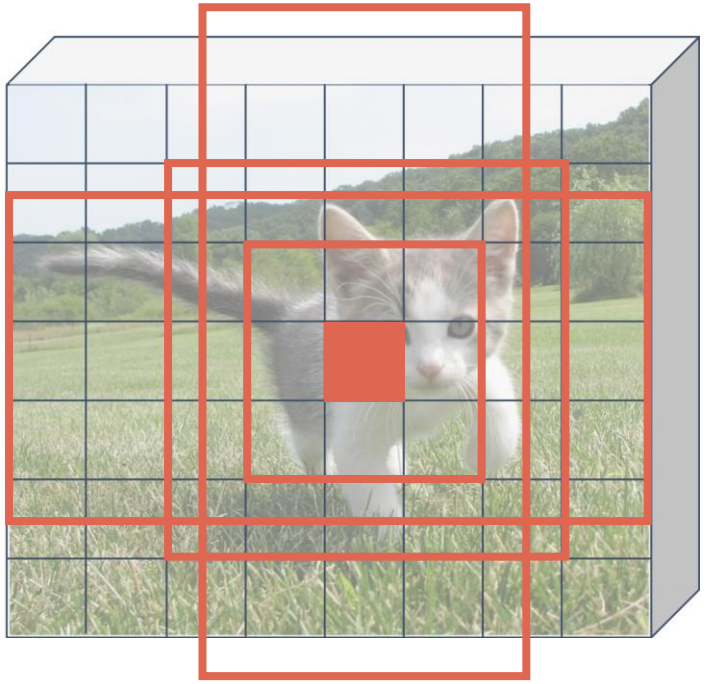


Image features
(e.g. 512 x 20 x 15)

Conv

Anchor category
(C+1) x K x 20 x 15

Box transforms
4K x 20 x 15

- K anchors at each position in image feature map

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Single-stage Object Detection

Run backbone CNN to get features aligned to input image



Input image
(e.g. 3 x 640 x 480)

CNN

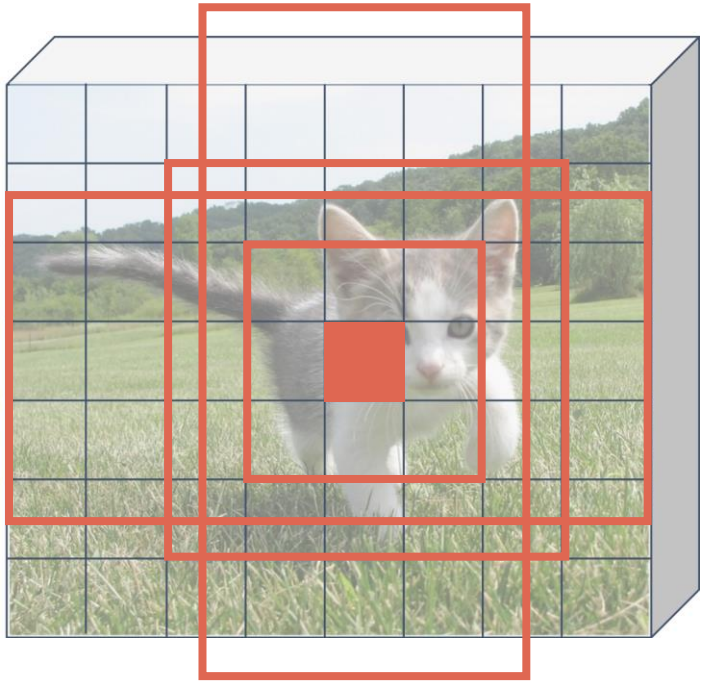


Image features
(e.g. 512 x 20 x 15)

Conv

- Anchor category
  (C+1) x K x 20 x 15
- Box transforms
  C x 4K x 20 x 15

- RPN: Classify each anchor as object / not object
- Single-stage detector: Classify each object as one of C categories (or background)
- Sometimes use category-specific regression: Predict different box transforms for each category

AALBORG UNIVERSITY
DENMARK

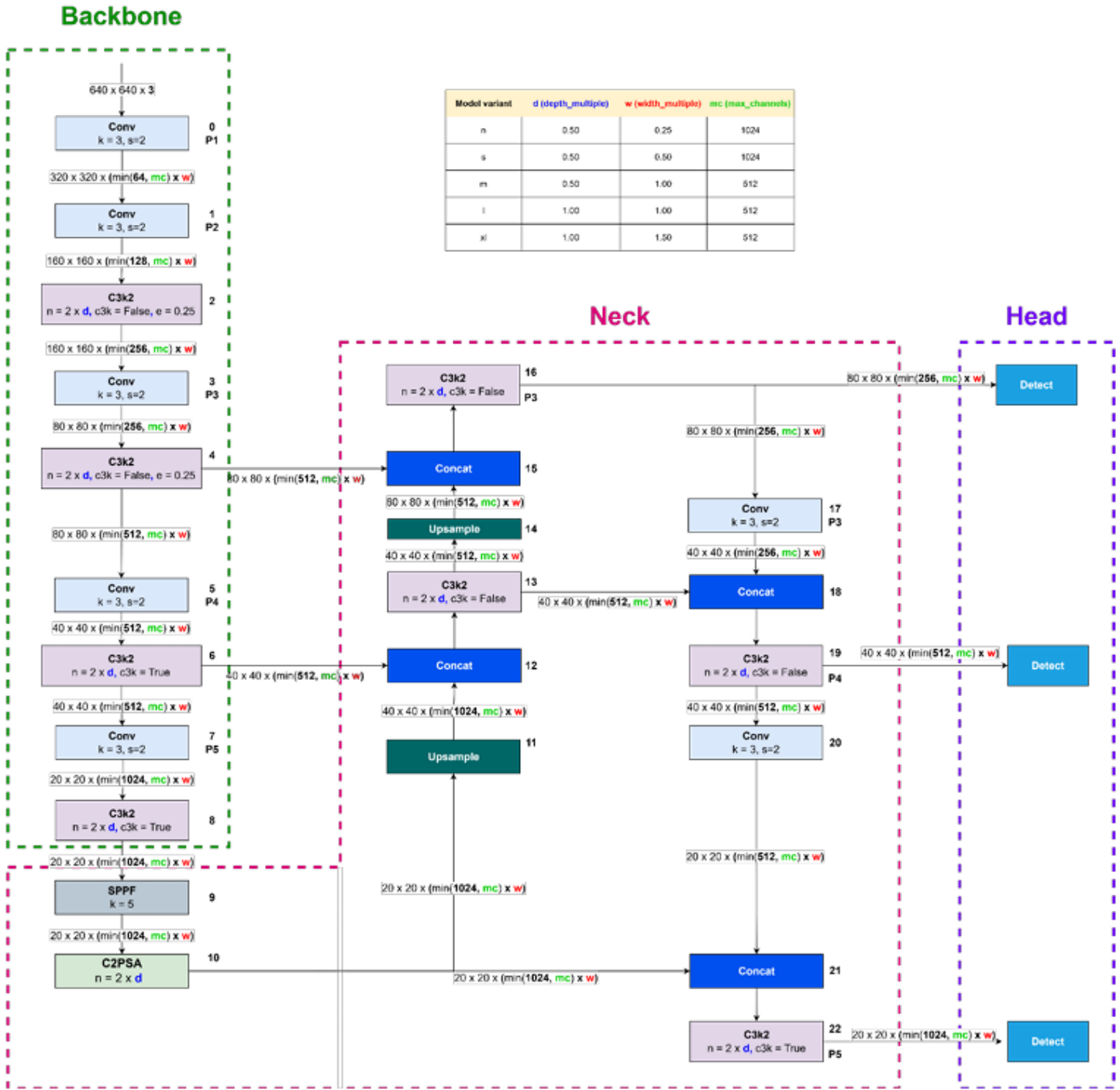
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Single-stage Object Detection

- Classical single-stage architectures:
  - YOLO (v1, v2, …, v12, v26)
  - SSD
  - …

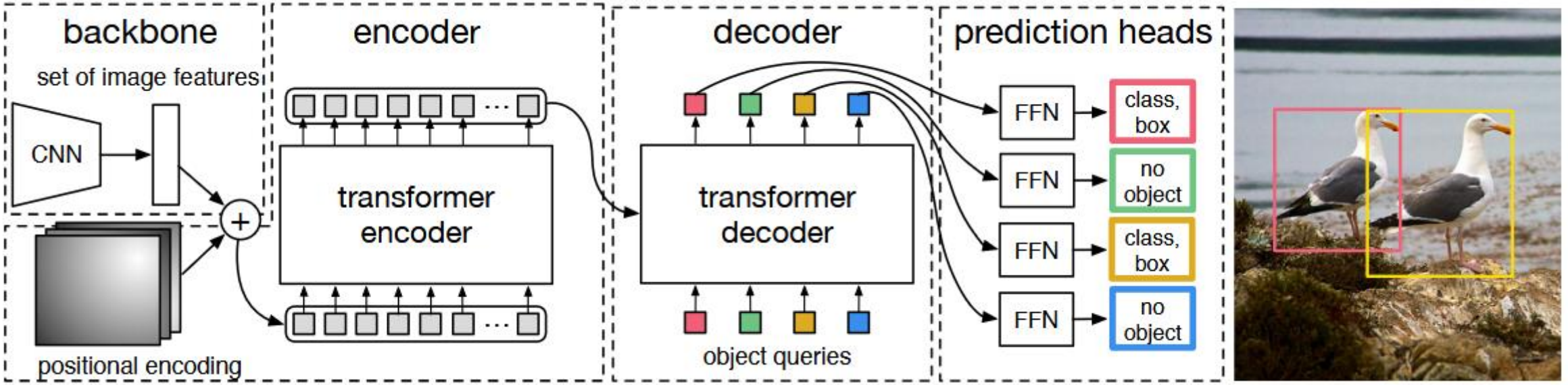
- Conventional thoughts:
  - Two-stage methods are more accurate, but slower
  - Single-stage methods are faster, but less accurate
- Today:
  - Single-stage methods are faster, and sometimes more accurate!

YOLO-v11



Model variant	d (depth_multiple)	w (width_multiple)	mc (max_channels)
n	0.50	0.25	1024
s	0.50	0.50	1024
m	0.50	1.00	512
l	1.00	1.00	512
xl	1.00	1.50	512

Tranformer based detection

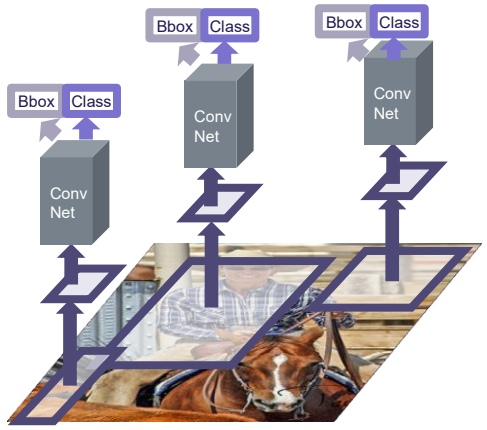


End-to-End Object Detection with Transformers (DETR), arxiv.org/pdf/2005.12872

Summary

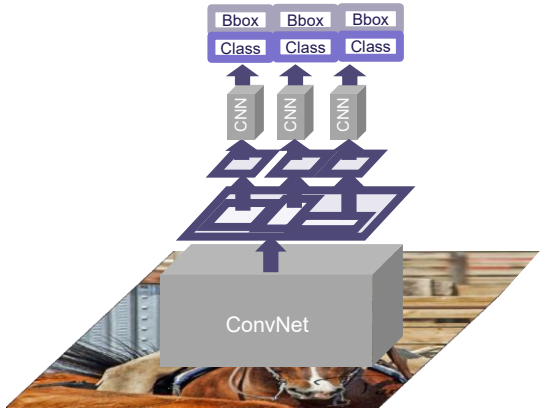
R-CNN

Run CNN independently for each region



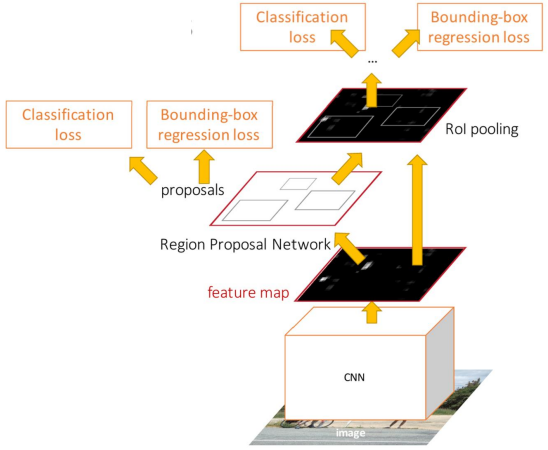
Fast R-CNN

Use shared image features



Faster R-CNN

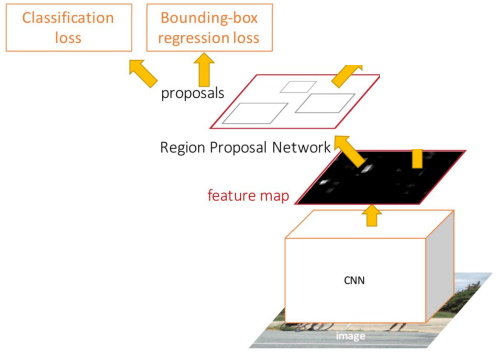
Compute region proposals with CNN



Single-stage

Fully convolutional detector

SOTA CNN-based detection today



Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Semantic segmentation

Pixel segmentation

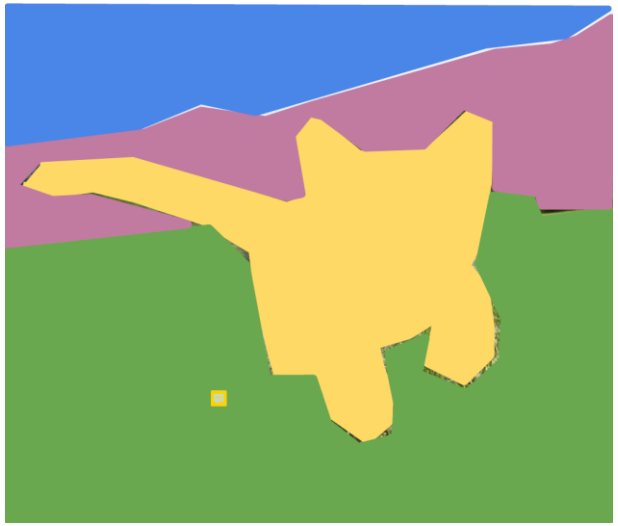
Classification



CAT

No spatial extent

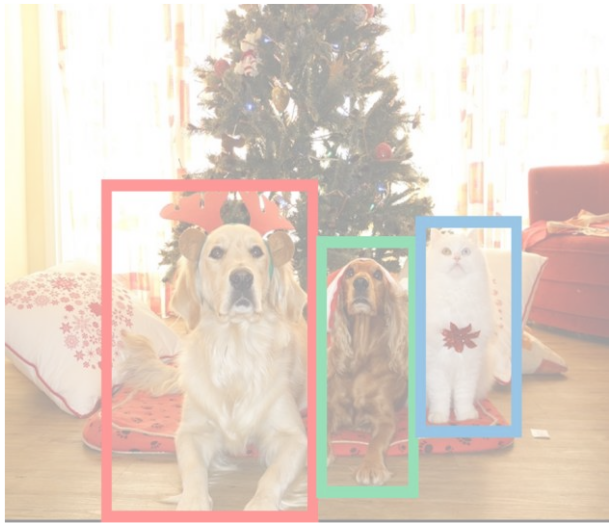
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object detection



DOG, DOG, CAT

Instance segmentation



DOG, DOG, CAT

Multiple objects

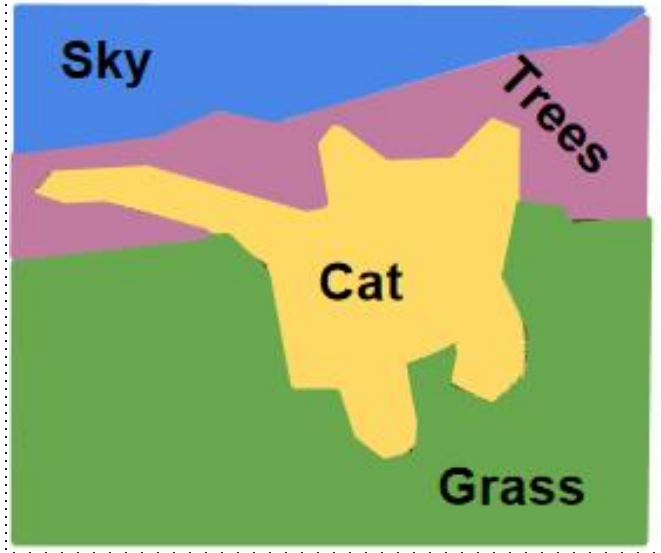
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

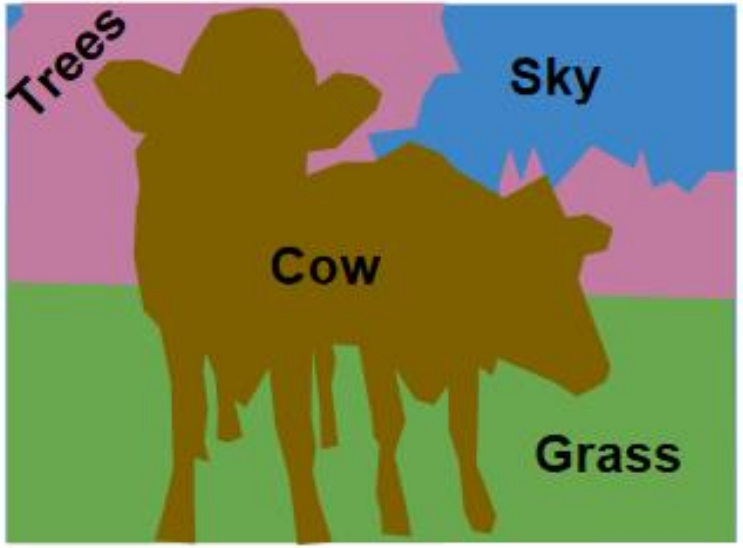
Semantic segmentation

- Label each pixel in the image with a category label
- Don’t differentiate instances, only care about pixels









Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

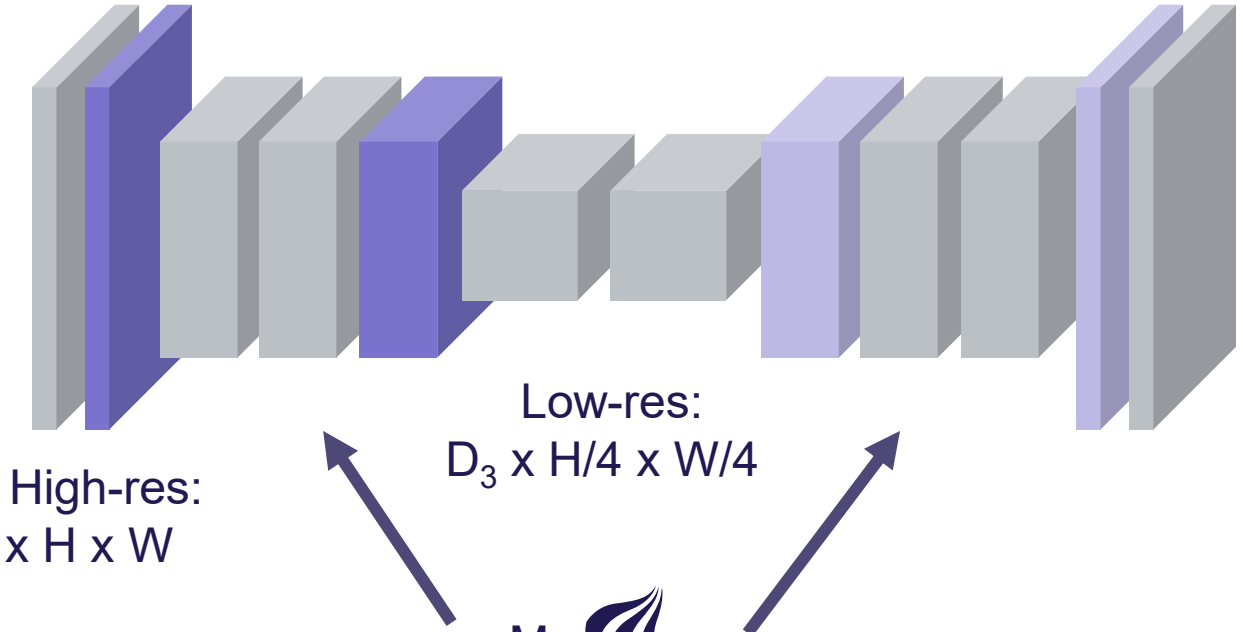
Semantic segmentation – Fully convolutional network

Downsampling:
- **Pooling**
- **Strided convolution**

Design network as a bunch of convolutional layers with downsampling and upsampling inside the network

Upsampling:
- **???**







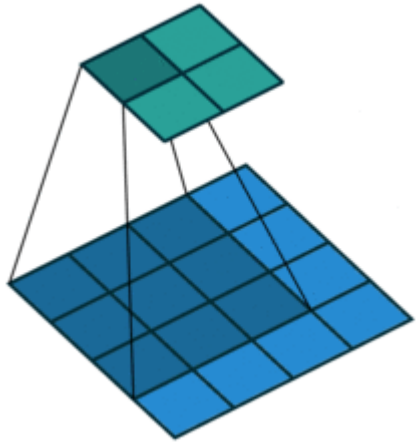
Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/

Learnable upsampling
- Transposed convolution

Regular convolution

Padding: 0

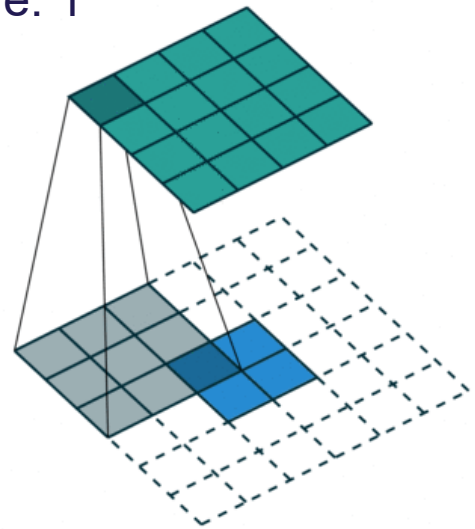
Stride: 1



Transposed convolution

Padding: 0

Stride: 1



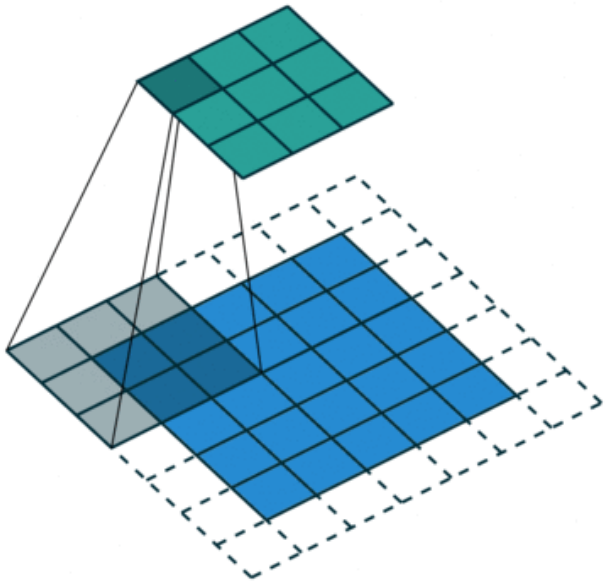
Conv illustrations: https://github.com/vdumoulin/conv_arithmetic

Learnable upsampling
- Transposed convolution

Regular convolution

Padding: 1

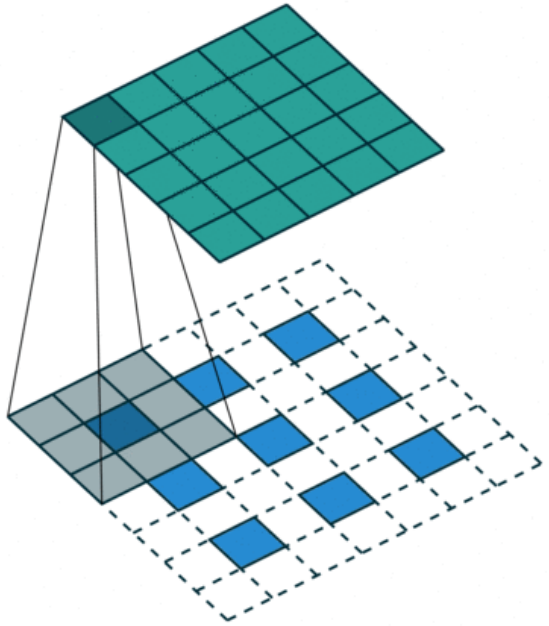
Stride: 2



Transposed convolution

Padding: 1

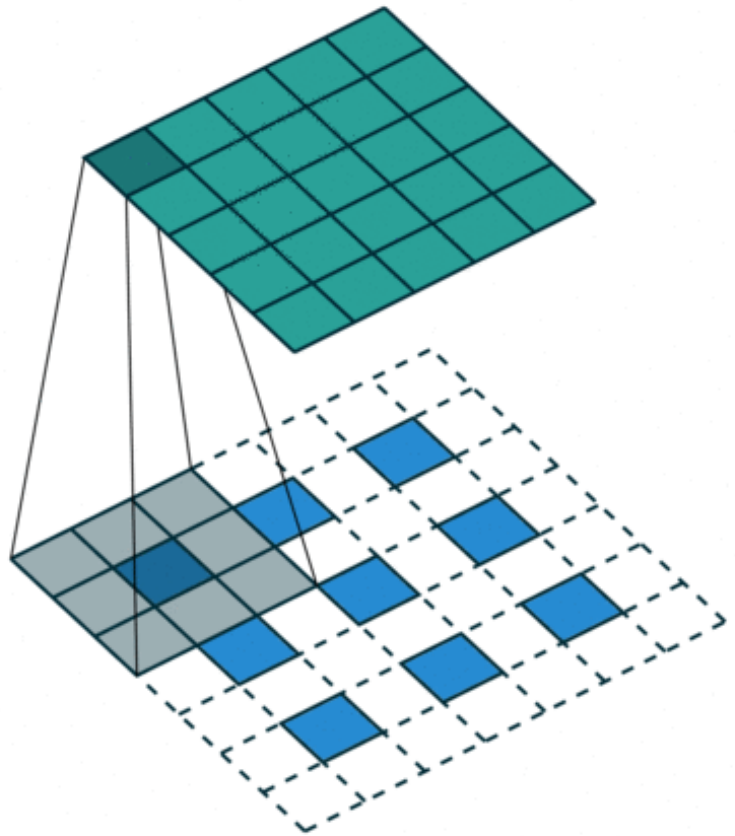
Stride: 2



Conv illustrations: https://github.com/vdumoulin/conv_arithmetic

Transposed convolution

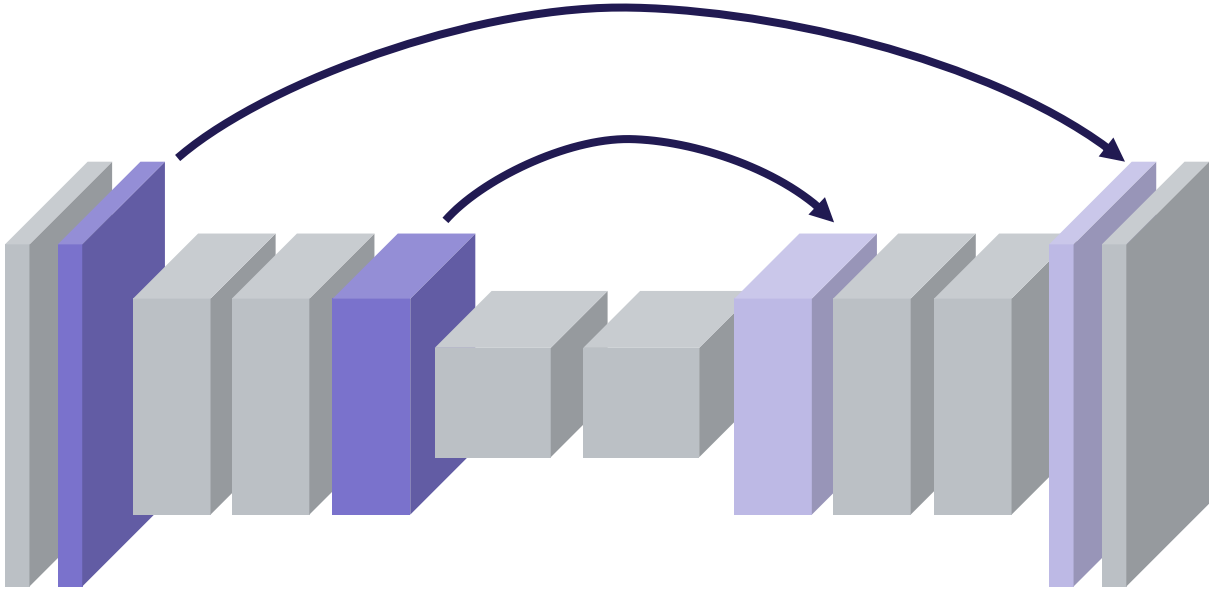
- Also known as:
  - Deconvolution
  - Upconvolution
  - Fractionally strided convolution
  - **Transposed convolution**



Conv illustrations: https://github.com/vdumoulin/conv_arithmetic

Semantic segmentation – Fully convolutional network

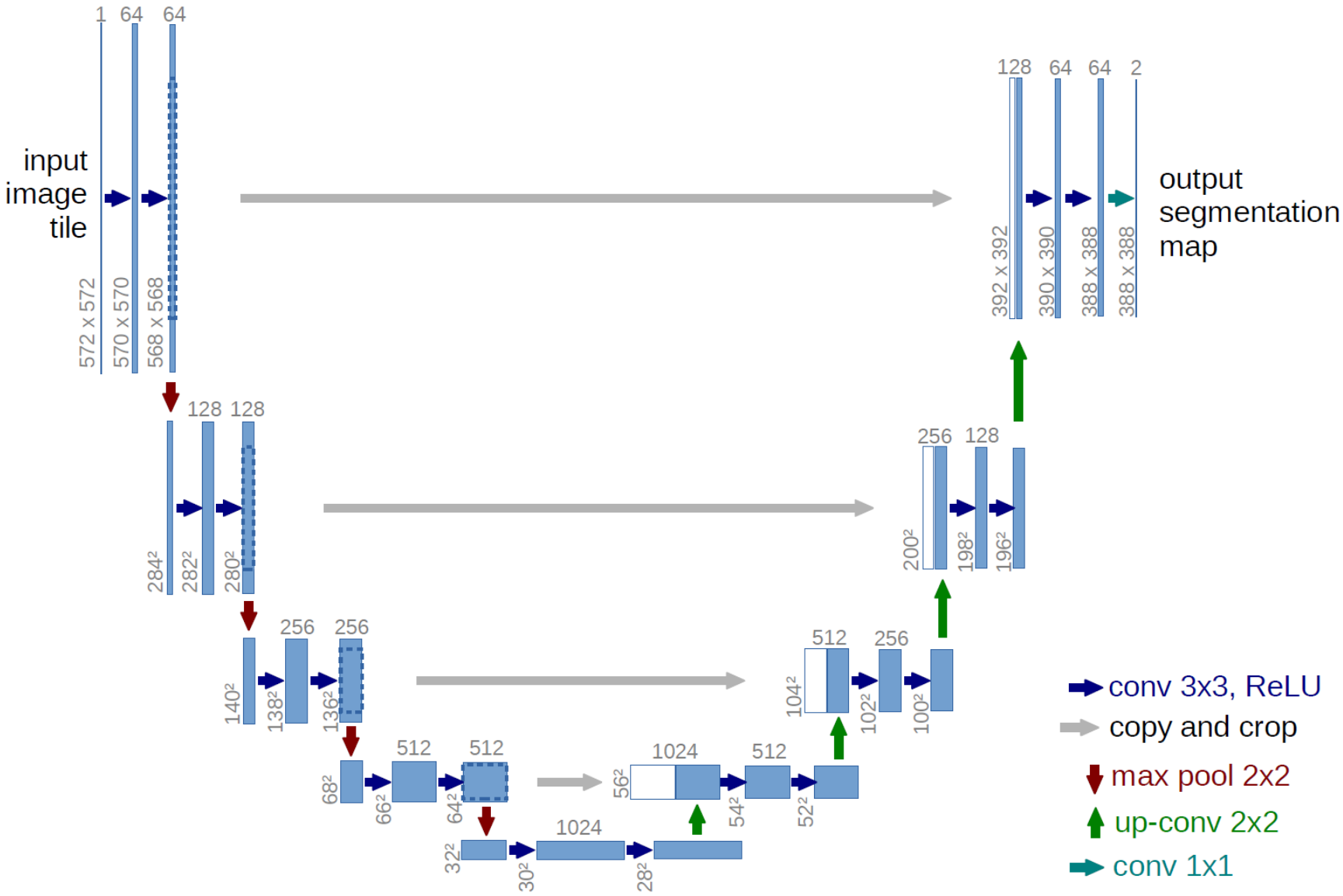






Add ”skip connections” between scales to preserve fine-grained textural information

U-net


input image tile
output segmentation map
1 64 64
572 x 572
570 x 570
568 x 568
128 128
284²
282²
280²
256 256
140²
138²
136²
512 512
68²
66²
64²
1024
32²
30²
28²
1024 512
56²
54²
52²
512 256
104²
102²
100²
256 128
200²
198²
196²
128 64 64 2
392 x 392
390 x 390
388 x 388
388 x 388
conv 3x3, ReLU
copy and crop
max pool 2x2
up-conv 2x2
conv 1x1

Instance segmentation

Segmentation vs detection

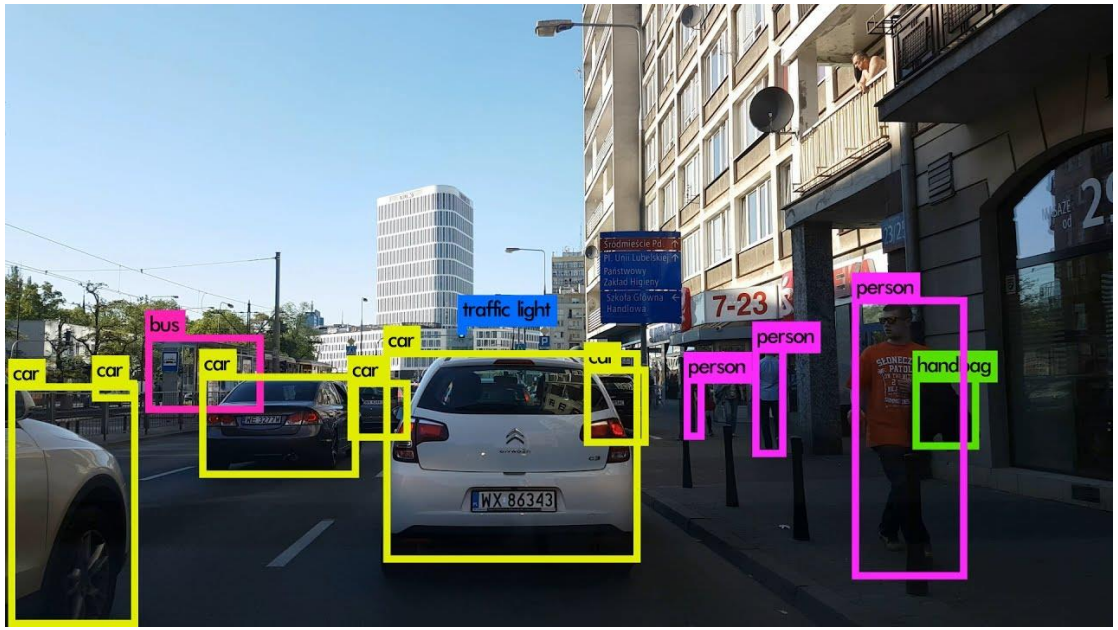
Segmentation

- Pixel-level labels
- Category only



Detection:

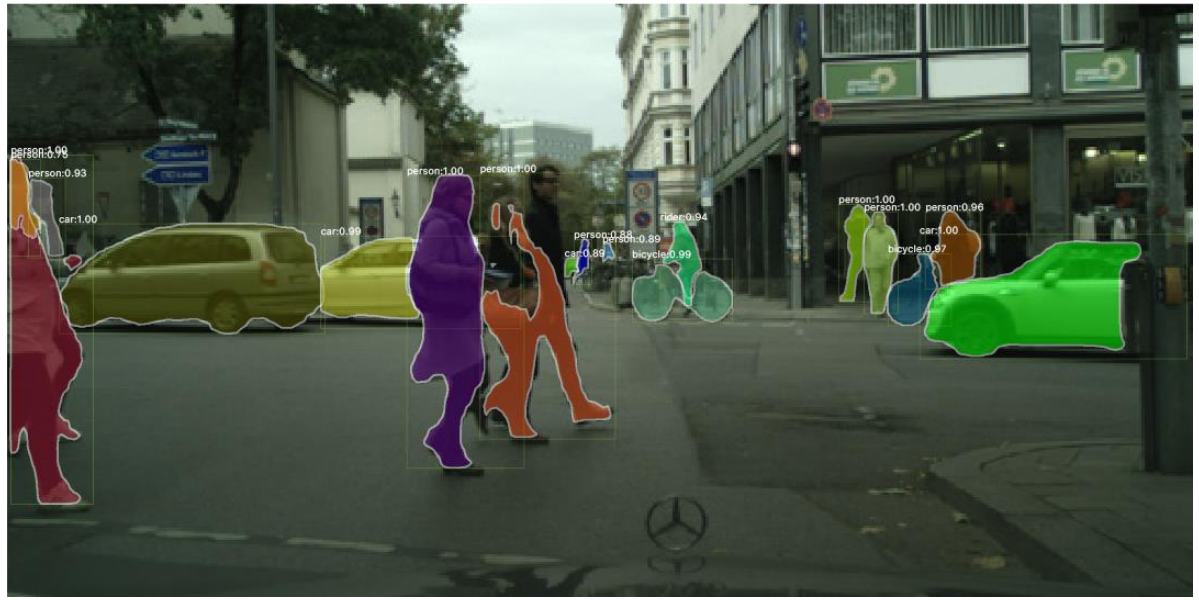
- Bounding box labels
- Category + instance

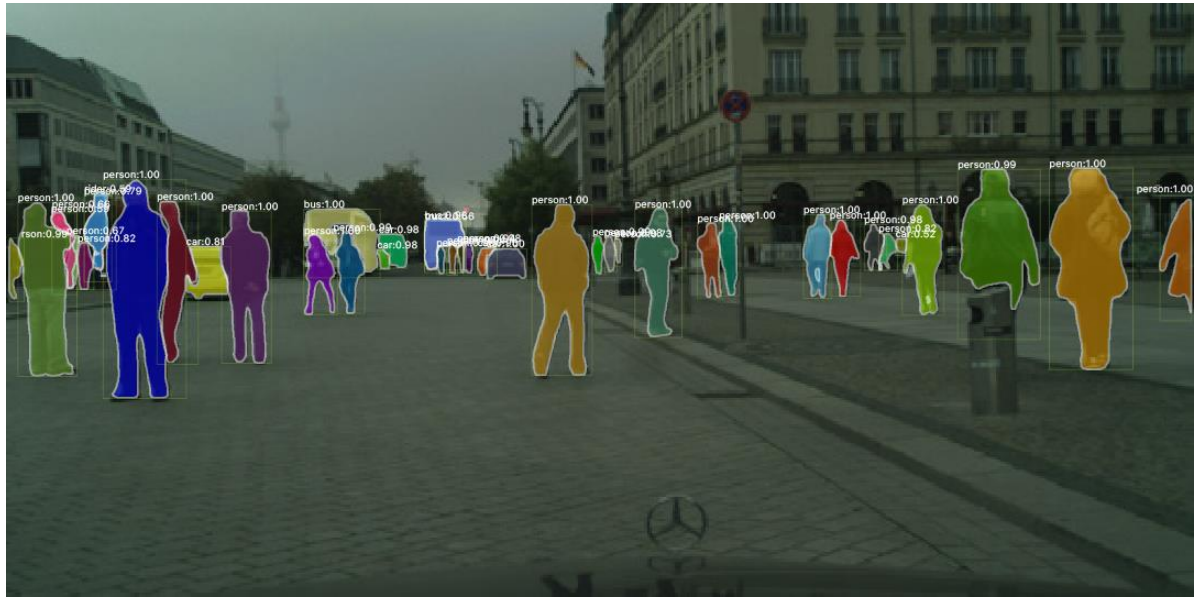


Instance segmentation

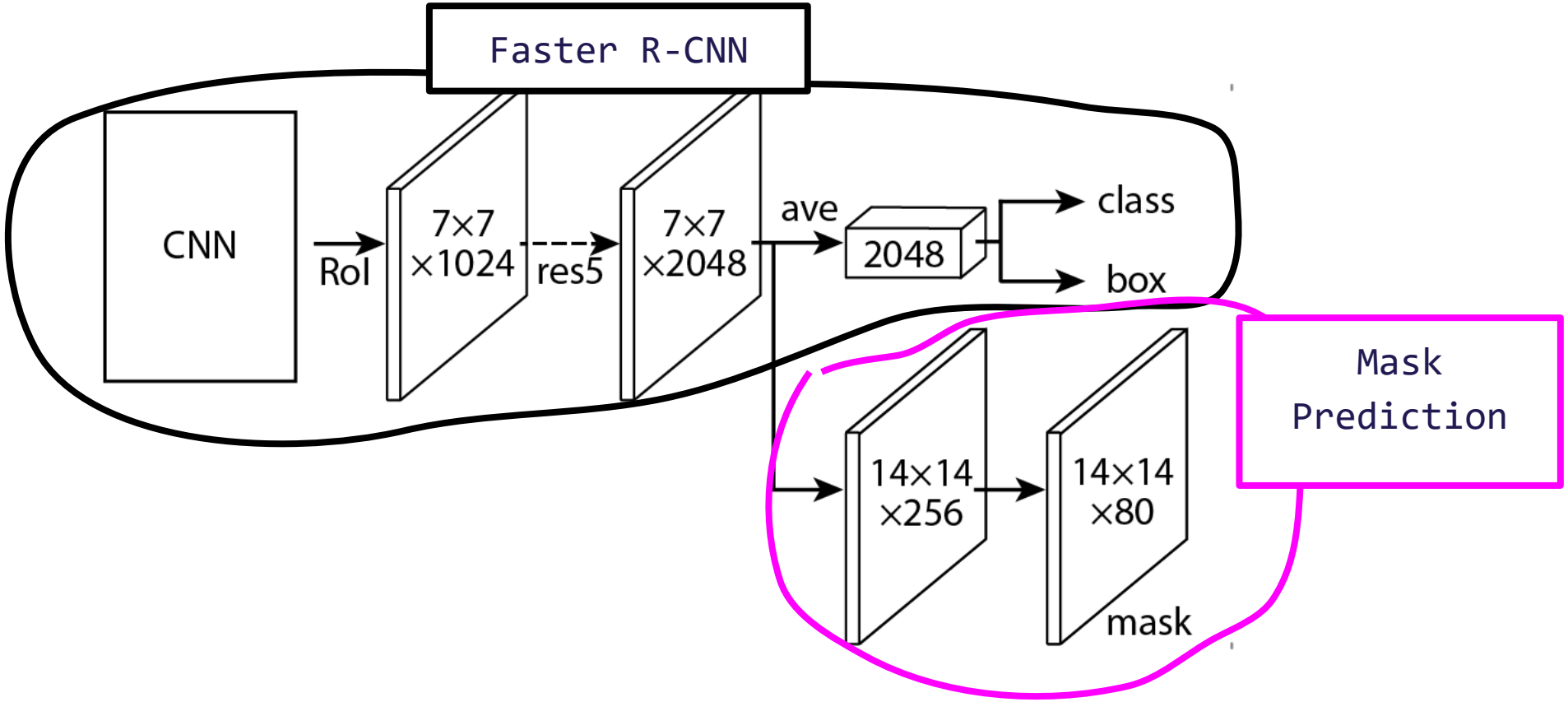
Given an image produce instance-level segmentation

- Which class does each pixel belong to
- Also which instance



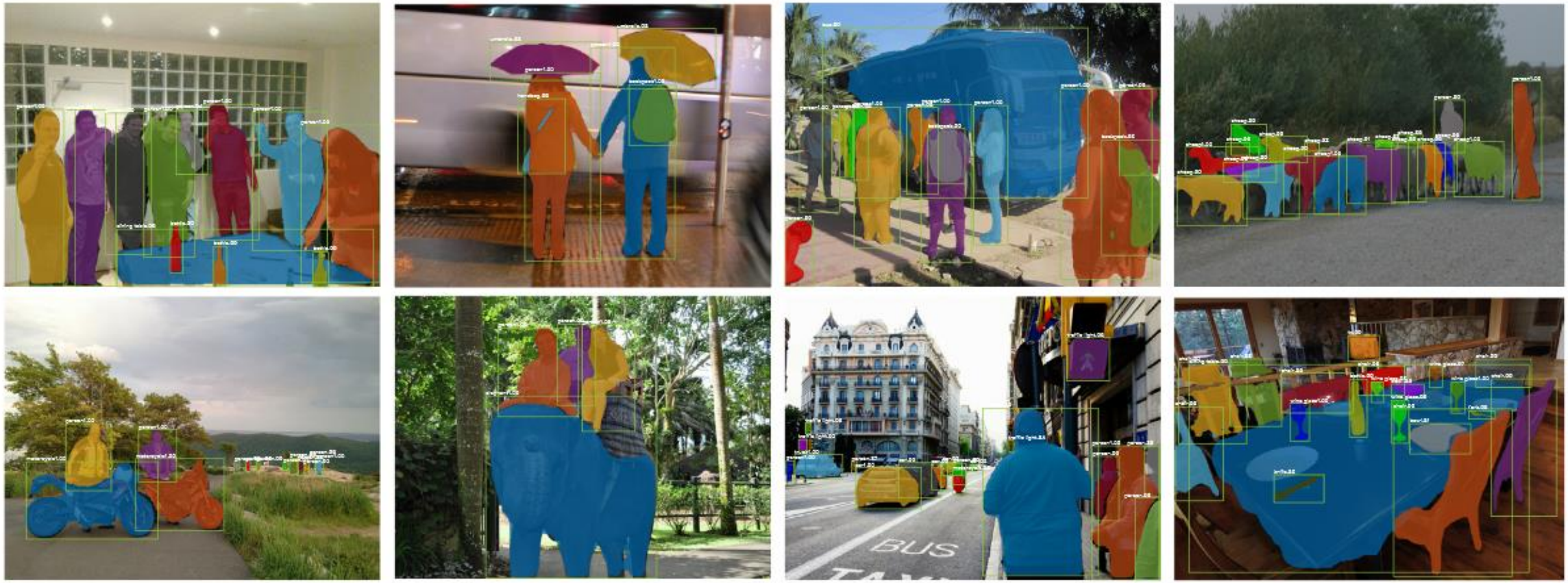


Instance segmentation – Mask R-CNN



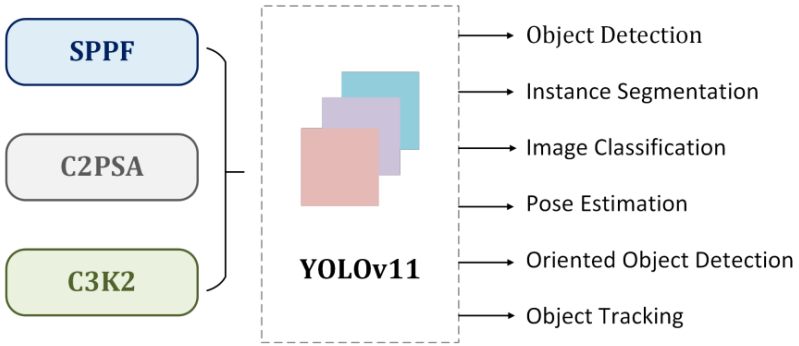
Mask R-CNN:
https://arxiv.org/pdf/1703.06870.pdf

Instance segmentation – Mask R-CNN



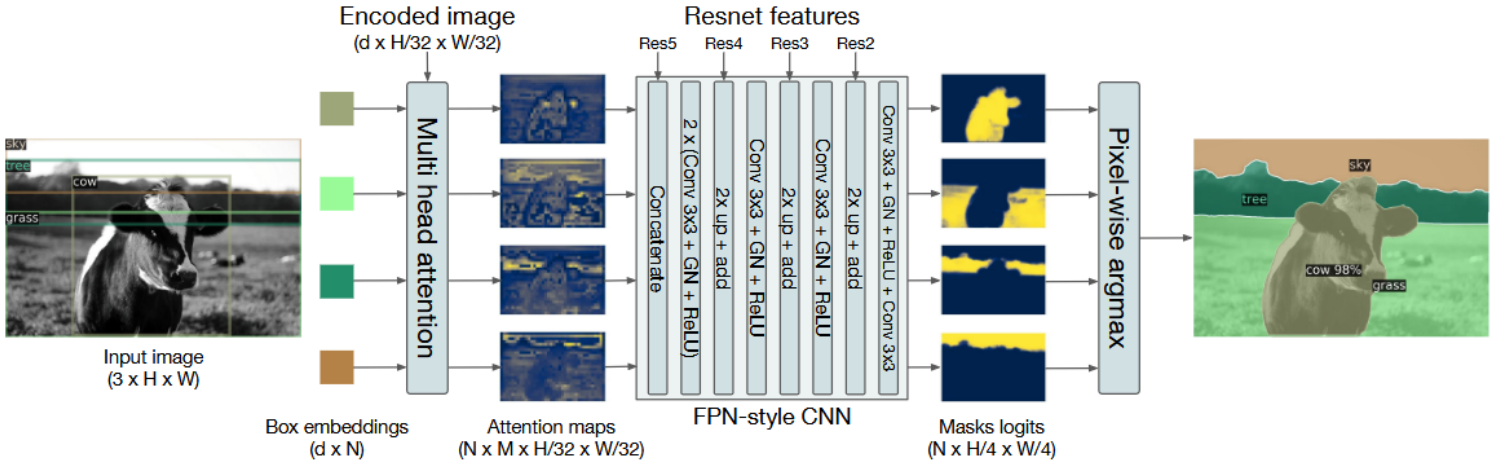
Mask R-CNN results on the COCO test set. These results are based on ResNet-101, achieving a mask Average Precision of 35.7 and running at 5 fps.
https://arxiv.org/pdf/1703.06870.pdf

Instance segmentation today



- YOLOv12
- YOLOv26
- RF-DETR

…

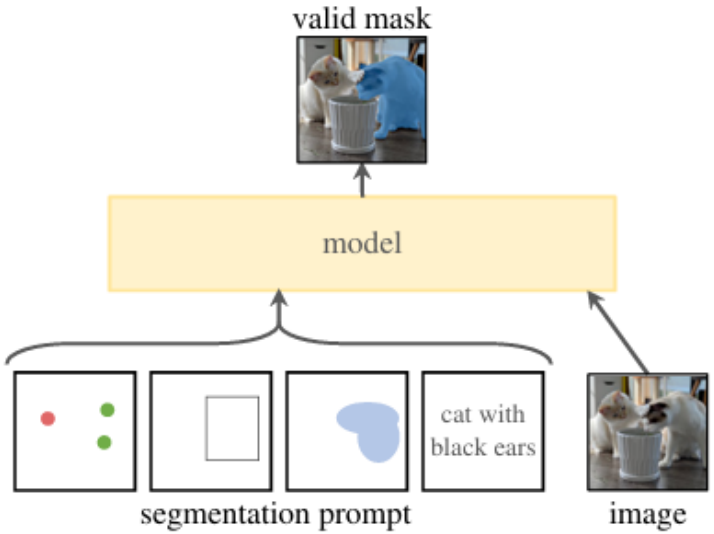




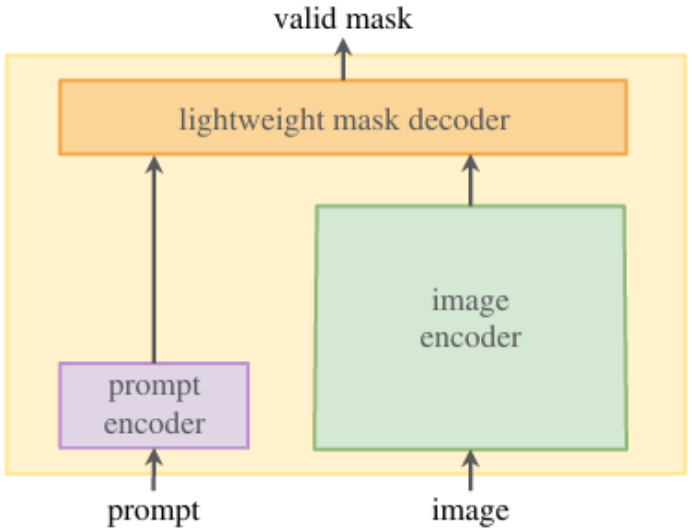
End-to-End Object Detection with Transformers (DETR), arxiv.org/pdf/2005.12872

Beyond classes (and CNNs): Promptable segmentation

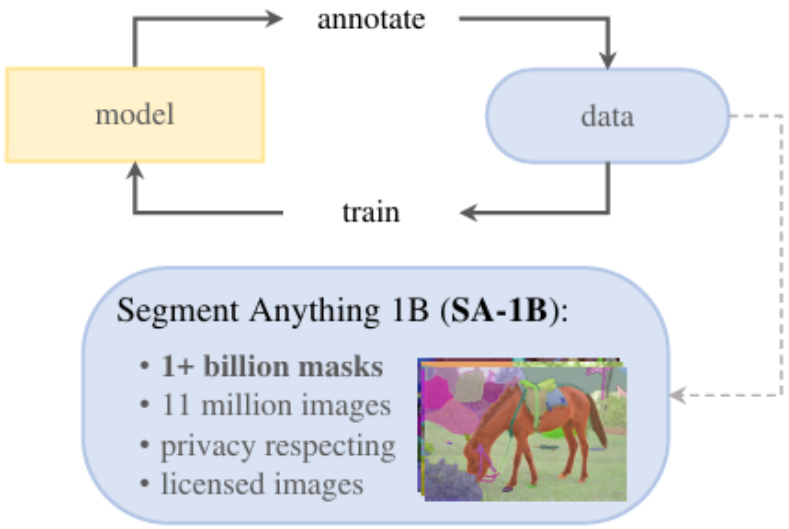
Segment Anything Project



(a) **Task**: promptable segmentation



(b) **Model**: Segment Anything Model (**SAM**)

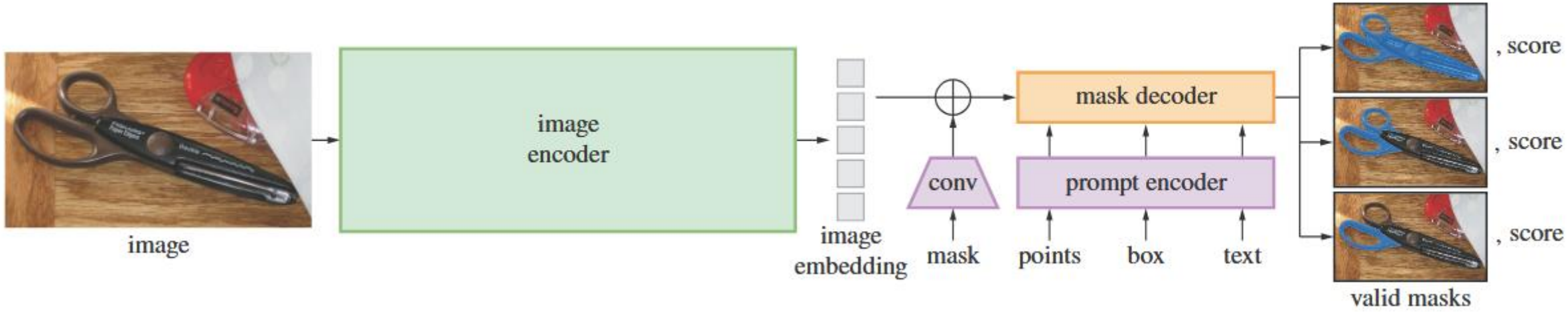


(c) **Data**: data engine (top) & dataset (bottom)

Segment Anything, https://segment-anything.com/
https://arxiv.org/abs/2304.02643

Segment Anything Model (SAM)

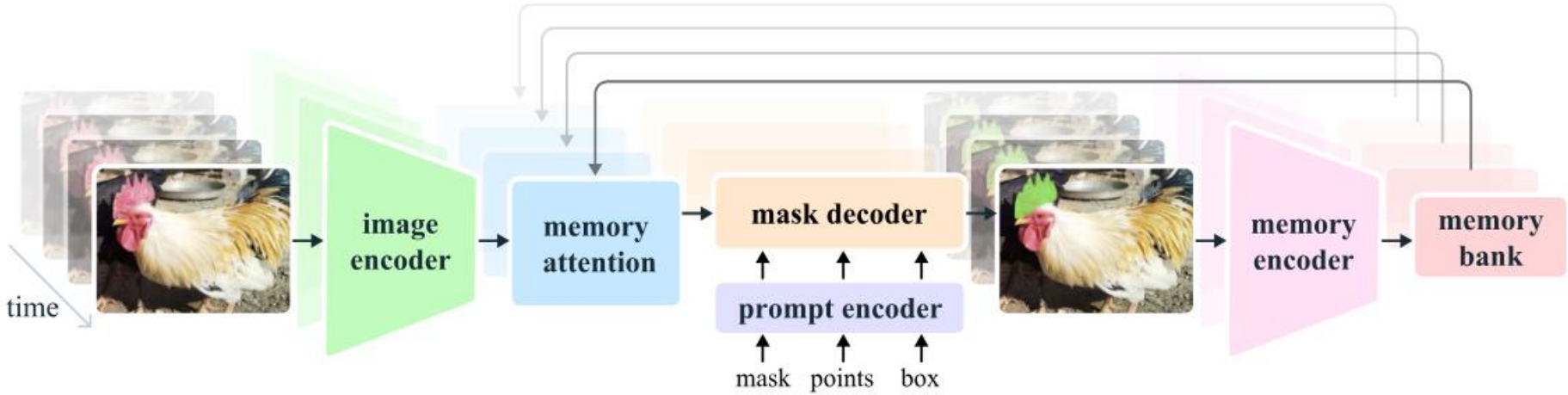
- Zero-shot segmentation
- Prompts: Points, box, text
- Encoders and decoders: Based on (Vision) Transformers



Segment Anything, https://segment-anything.com/
https://arxiv.org/abs/2304.02643

SAM 2

- From image to video



SAM 2: Segment Anything in Images and Videos
https://arxiv.org/abs/2408.00714

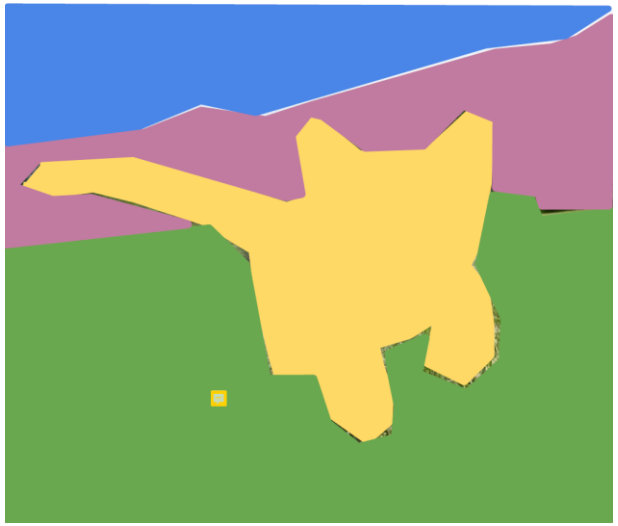
Summary

Image Classification



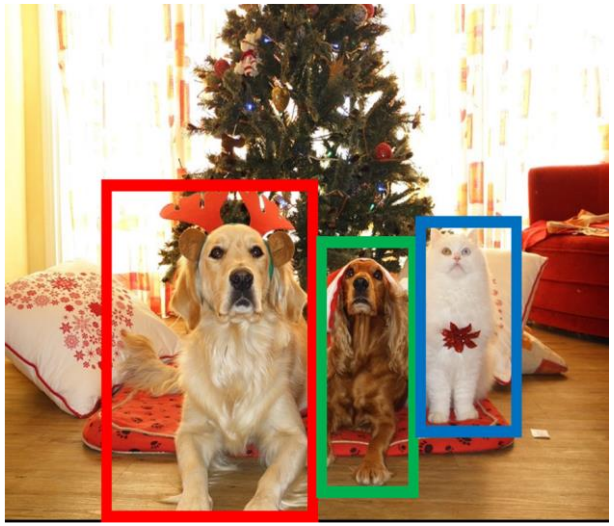
CAT

Semantic Segmentation



GRASS, CAT, TREE, SKY

Object detection



DOG, DOG, CAT

Instance segmentation



DOG, DOG, CAT

Adapted from Justin Johnson, https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/